

Serverseitige Entwicklung von Javascript-Apps mit Node.js

2026, Michael Reichart
GFU Cyrus AG, Köln

Wir bilden weiter.

2026 Michael Reichart, copyleft CC BY-NC-SA

2026 Michael Reichart, copyleft CC BY-NC-SA

9:00 Uhr bis 16:00 Uhr
Mittag 12:00 Uhr - 13:00 Uhr
10:30 und 14:30 je 15 Minuten Pause.

kollegial?

DU ?

formell?

SIE ?

2026 Michael Reichart, copyleft CC BY-NC-SA

GFU Cyrus AG, Köln

- Fortbildung für Unternehmen
- 600+ erfahrene Dozenten, die wissen, was sie tun.
- 6000+ Seminare aus den Bereichen Anwendung, Administration, Entwicklung, UX, Organisation und Kommunikation.
- Ein routiniertes und begeistertes Mitarbeiterteam.
- Shuttle- und Hotelservice und die beste Candybar, die es je in einem Seminarzentrum gegeben hat.

2026 Michael Reichart copyleft CC BY-NC-SA

Michael Reichart

Designer, Coder, Coach

- Gestalter für visuelle Kommunikation
- Softwareentwickler für Browser-/Server-Software
- 1996 - 2017 CEO der Digitalwerkstatt Stuttgart
- Seit 2016
  member of strategy board
  GFU Cyrus AG
- Seit über 20 Jahren Dozent und Coach
  für Unternehmen und an Hochschulen.



2026 Michael Reichart, copyleft CC BY-NC-SA

SEMINARE FÜR ...

- **HTML5 (HTML, CSS, Javascript)**
- **Webapplikationen mit nodejs**
- Softwarearchitektur
- **Barrierefreiheit** in der IT
- UX/UI Konzeption
- Physical Computing (C++, Microcontroller)
- Datenvisualisierung



2026 Michael Reichart, copyleft CC BY-NC-SA

- Auf LinkedIn unter Michael Reichart.
- Per Mail: michael.reichart@gfu.net
- Workshop-Codes auf https://github.com/zenbox/workshop



2026 Michael Reichart, copyleft CC BY-NC-SA



Wer bist Du und was ist deine Aufgabe im Unternehmen?

Wie schätzt Du deinen Kenntnisstand in der **HTML und CSS** ein?

Nichts

Sehr wenig, ich bin definitiv **Anfänger.**

Ich habe **bereits Erfahrungen**, aber das reicht nicht.

Ich kenne mich schon gut aus, möchte aber **besser verstehen.**

Ich glaube, ich weiss schon alles!

0 — 1 — 2 — 3 — 4 — 5 →

keine Erfahrung

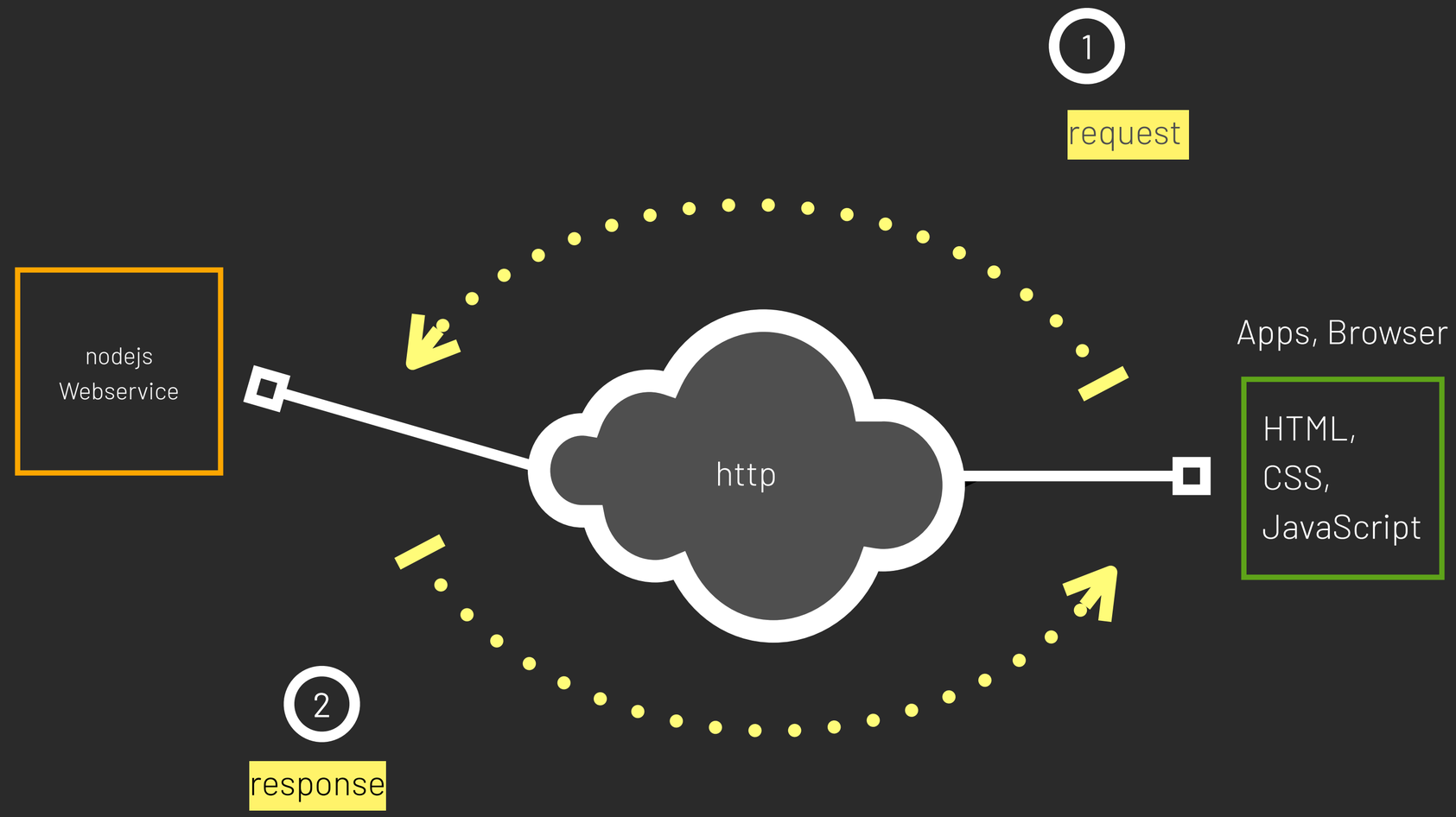
viel Erfahrung

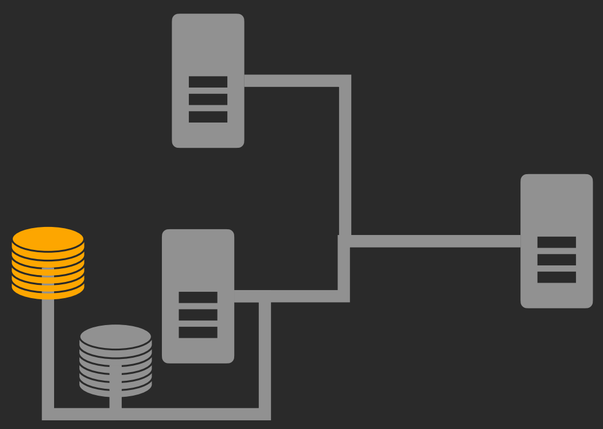


2026 Michael Reichart, copyleft CC BY-NC-SA

Skizzen

Schritt 1



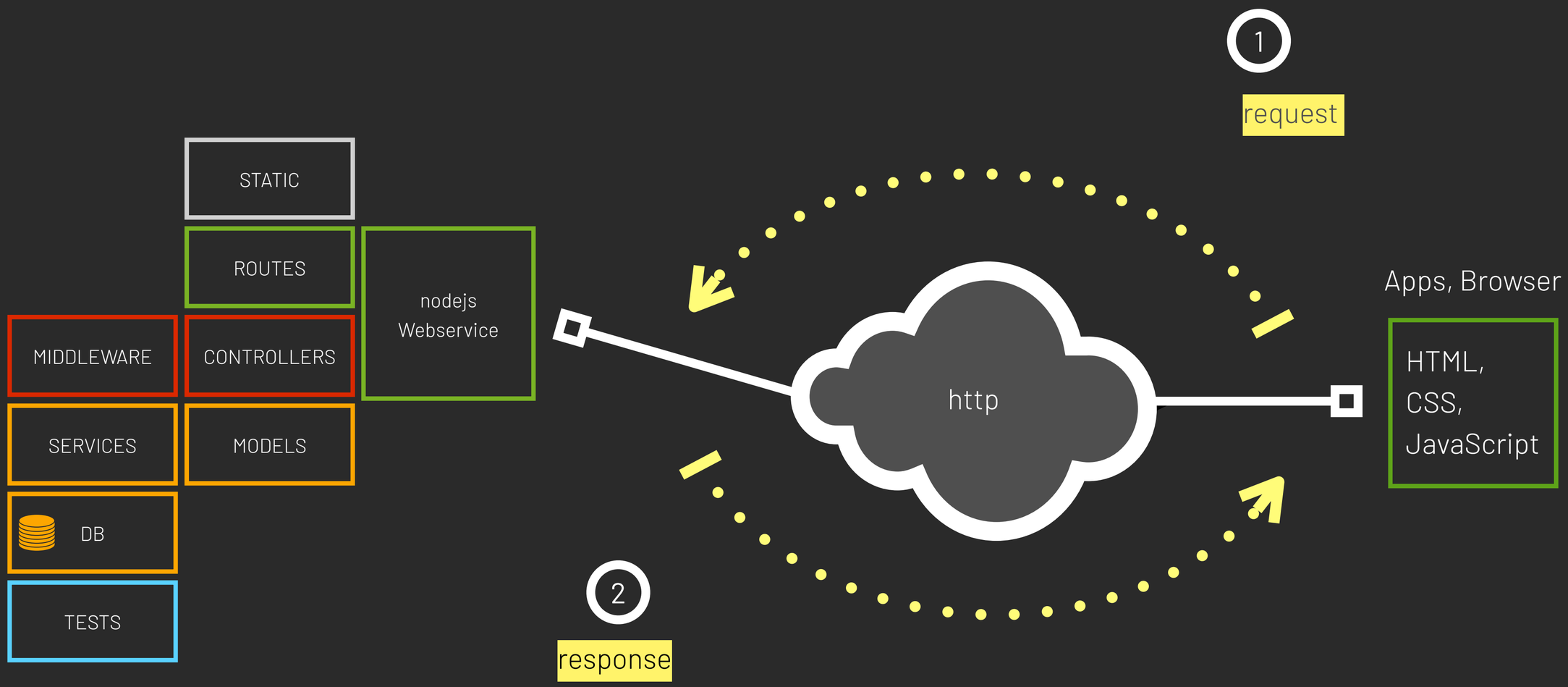


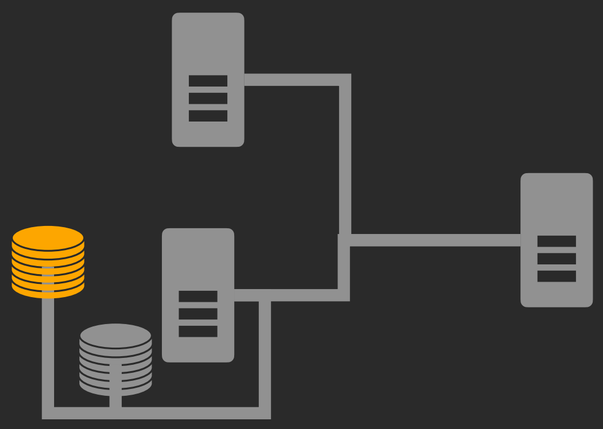
Server - Web - Client



2026 Michael Reichart, copyleft CC BY-NC-SA

Schritt 2





Server - Web - Client



2026 Michael Reichart, copyleft CC BY-NC-SA

DATABASE
MSSQL

DATABASE
SQLite

MODEL
User class

MIDDLE-
WARE A

MIDDLE-
WARE B

SASS

STATIC
z. B.
assets

CONTROLLER

classes or functions,
business logical stuff

VIEWS (Templates)

z. B. HTML
+ PUG, + EJS

ROUTES

Request-Handling

router.get("/" , getLoginPage
)
router.post("user/")

APP

HTTP(S)-Server
Websocket-Server

2

response

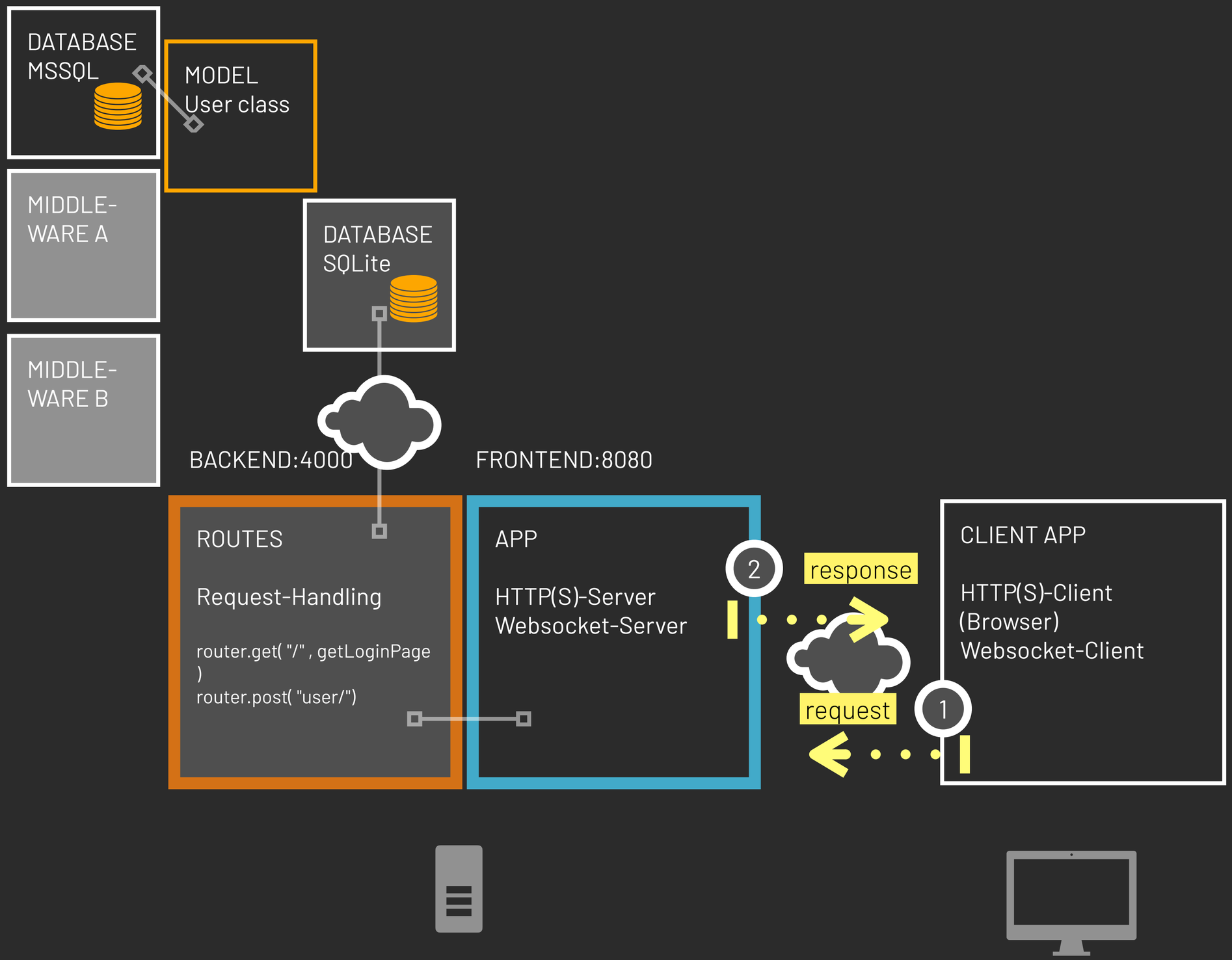
request

1

CLIENT APP

HTTP(S)-Client
(Browser)
Websocket-Client

2026 Michael Reichart, copyleft CC BY-NC-SA



2026 Michael Reichart, copyleft CC BY-NC-SA



PUT ist idempotent:
die mehrfache Anwendung einer Operation ändert das Ergebnis NICHT, nachdem sie einmal angewendet wurde.
PATCH ist wiederholbar.

CONTROLLER

ROUTES

APP

sheep-controller as class

onRequest()

DELETE

PUT PATCH

POST

GET

/

delete()

update()

save()

get()

sheep.js (Route, Controller, View)

/sheeps

DELETE

PUT / PATCH

POST

GET

index.html

2026 Michael Reichart, copyleft CC BY-NC-SA

CONTROLLER

ROUTES

APP

DELETE

PUT (PATCH)

POST

GET

OPTIONS

ReST-controller

getPosts()

login()

createPost()

patchPost()

deletePost()

/

/posts

/login

/post

/put/:id?

/delete/:id?

rest.js

/rest

broker-controller

onRequest()

/

broker.js)

/broker

frontend-controller

onHome()

/

frontend.js

/

/login

onLogin()

/index.html

/index.html

index.html

2026 Michael Reichart, copyleft CC BY-NC-SA

Projektumsetzung

Execution Stack, Threads and Eventloop

Non blocking I/O, Event Loops, Single Threads

2026 Michael Reichart, copyleft CC BY-NC-SA

ECMA
European Computer Manufacturer Association

ES 17
ECMA Script

common parts:
simple and enhanced variables,
object constructors,
strict coding rules

global unnamed object		
Array	Object	Function
Date	Math	RegExp
JSON	...	
String	Number	Boolean

CLIENTSIDE

nodejs SERVERSIDE
as library

Javascript
as library

Mozilla Foundation

Adobe

Node.js

Deno

2003/2004: ES 5

Document Object Model (Browser DOM)

PDF Scripting

nodejs - Lib

deno - lib

	window		
objects	document	navigator	console
	frames [ ]	...	...
	images[ ]	...	
	...		
methods	querySelector()	...	...
	addEventListener()		
	...		
values	screenWidth	online	...
	...	...	...

process			
http / https	filesystem	console	...
socket	...	...	

2026 Michael Reichart, copyleft CC BY-NC-SA

BUTTON

click

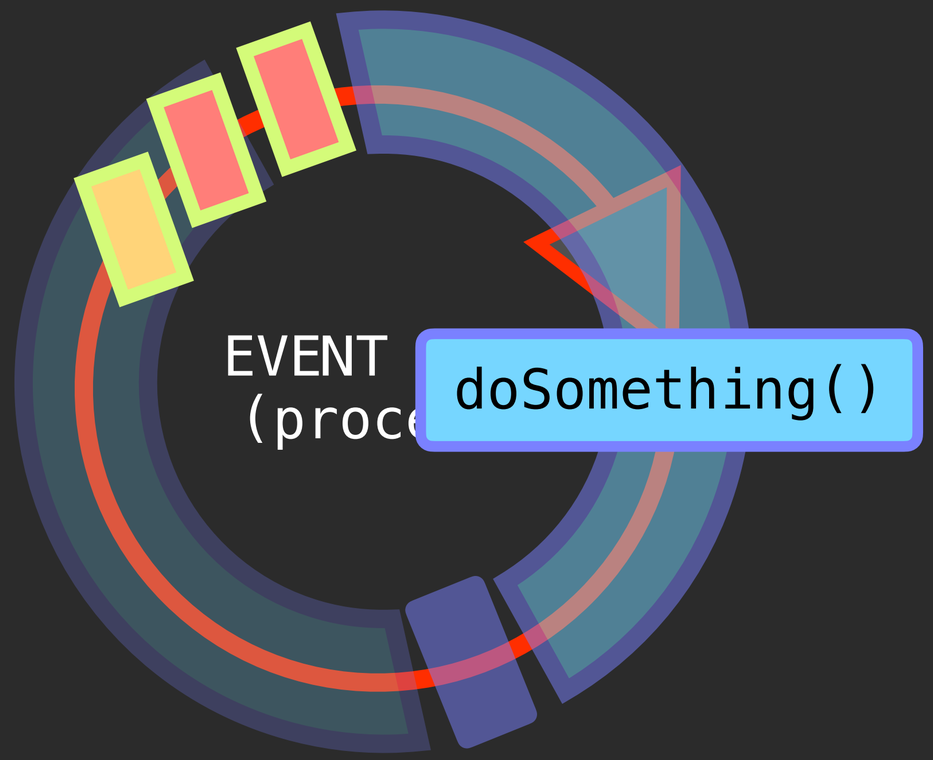
OBJECT

load

submit

message

EVENT STACK
(internal event bus)



callback()

other()

await fetch()

send()

doSomething()

THREADPOOL
(fn literals)

callback()

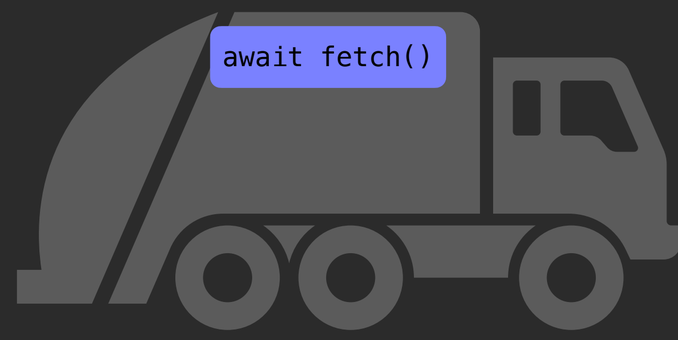
send()

ASYNC MICROTASKS

EXECUTION STACK
(lifo – last in first out)

to stack
listen()

call()



2026 Michael Reichart, copyleft CC BY-NC-SA

```
10 REM Mein zweites Programm
20 GOTO 40
30 END
40 LET result = 1 + 1
50 IF result = 2 THEN GOTO 80
60
70
80 END

10000 DATA 1,2,3,4,5
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Installation und Konfiguration

From the scratch

2026 Michael Reichart, copyleft CC BY-NC-SA

node

24.21.0 LTS

Recommended For Most Users

26.10.0 Current

Latest Features

Run JavaScript Everywhere

Node.js® is a free, open-source, cross-platform JavaScript runtime environment that lets developers create servers, web apps, command line tools and scripts.

Get Node.js®

Get security support

for EOL Node.js versions



Node.js is proudly supported by the partners above and more.

Create an HTTP Server | Write Tests | Read and Hash a File | Streams Pipeline

```mjs
// server.mjs
import { createServer } from 'node:http';

const server = createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' })
  res.end('Hello World!\n');
});

// starts a simple http server locally on port 3000
server.listen(3000, '127.0.0.1', () => {
  console.log('Listening on 127.0.0.1:3000');
});

// run with `node server.mjs`
```

JavaScript | Copy to clipboard

Learn more what Node.js is able to offer with our Learning materials.

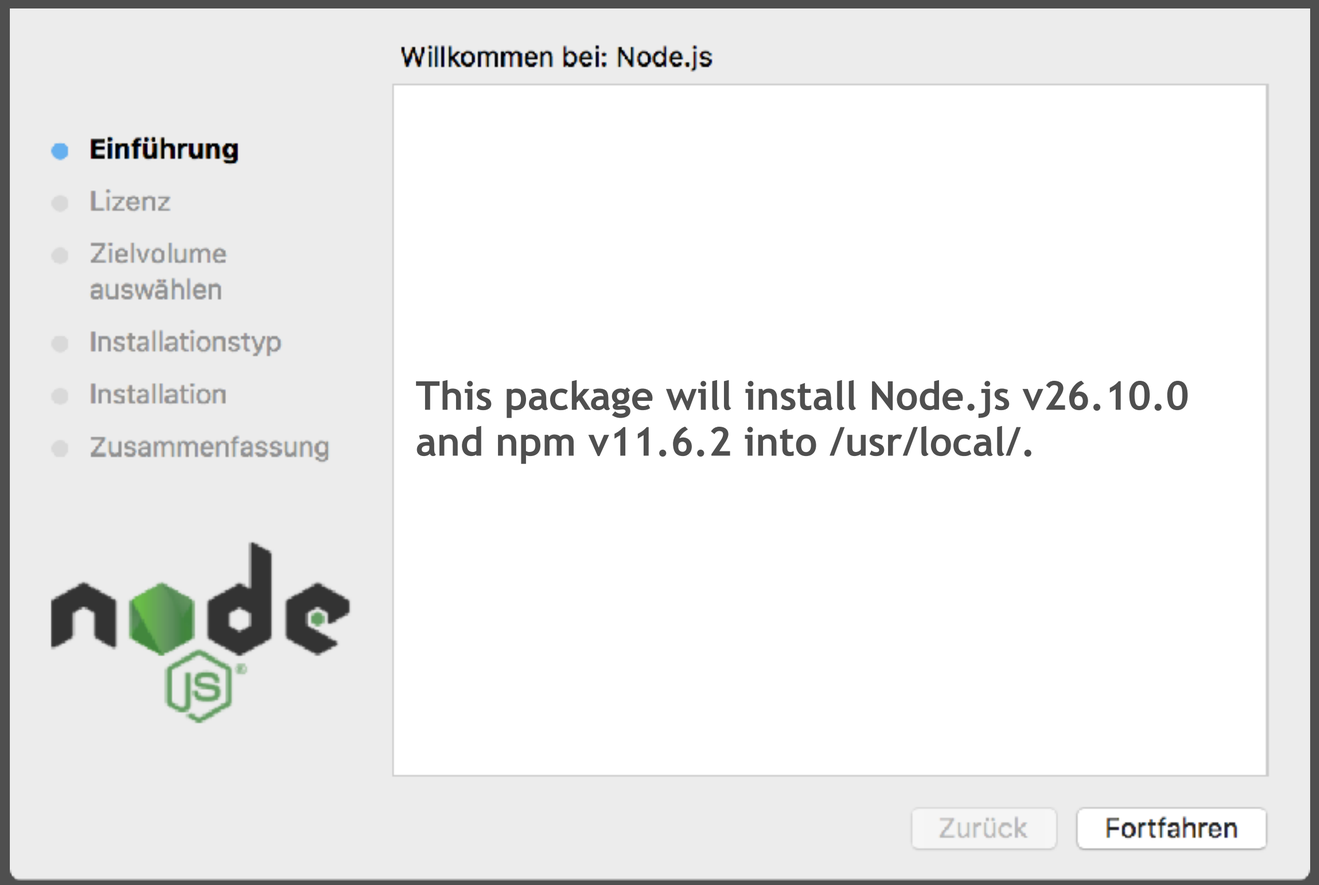
2026 Michael Reichart, copyleft CC BY-NC-SA

+ Installer für Windows und Mac OS

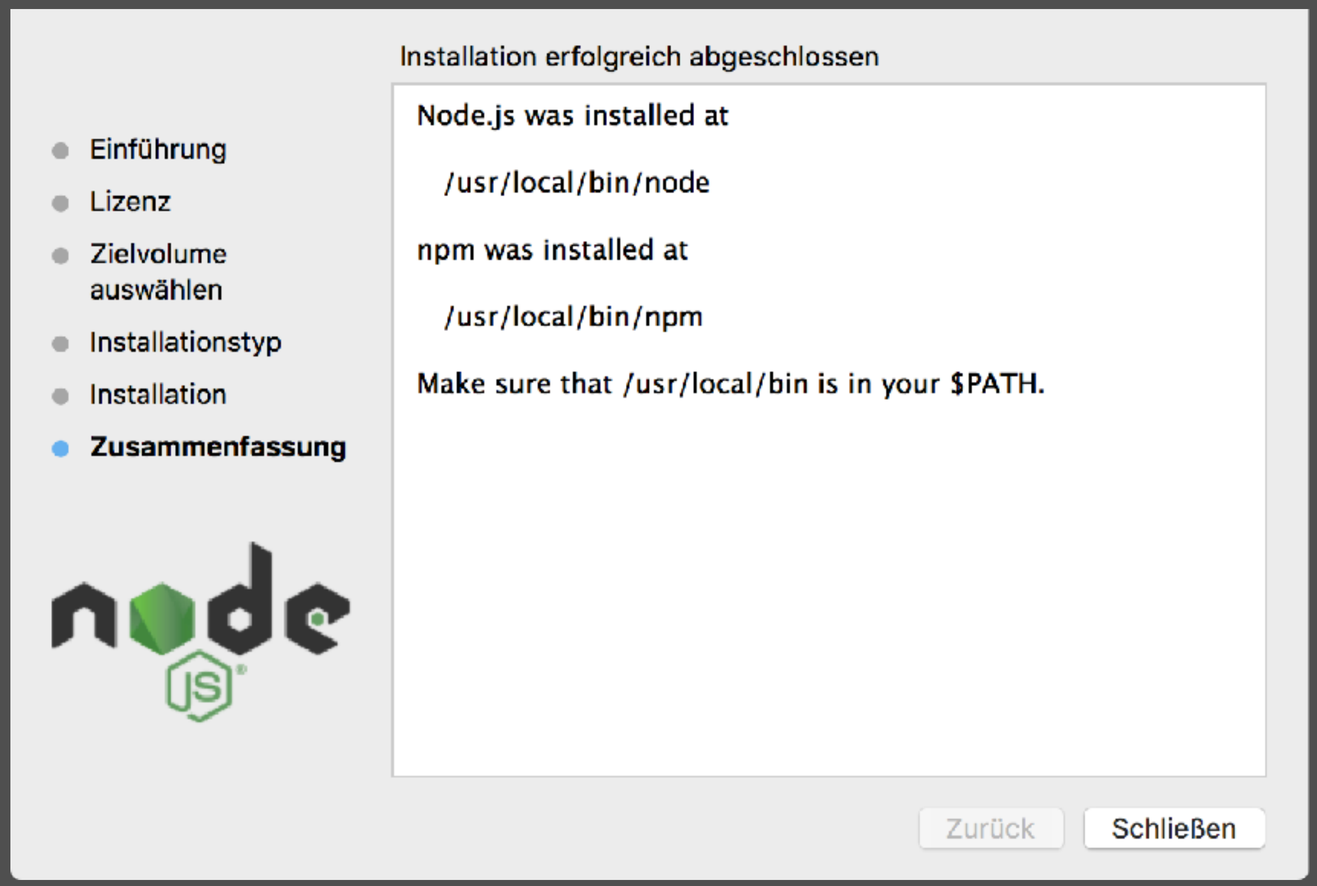
+ Erweiterbar über Node Package Manager

+ geschrieben in C++, individuell kompilierbar

2026 Michael Reichart, copyleft CC BY-NC-SA



2026 Michael Reichart, copyleft CC BY-NC-SA



2026 Michael Reichart, copyleft CC BY-NC-SA

Node abfragen

```
$ node -v
v26.10.0

$ npm -v
V11.6.2
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Nodejs via "n" im Terminal aktualisieren

```
[$ npm cache clean -f]
$ npm install -g n
$ n stable
$ n latest
$ n 11.6.2
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Alternativ: nvm verwenden

```
nvm ls
nvm install 20
nvm install 11.6.2
nvm use 20
nvm alias default 20
nvm install node
nvm use node
node -v
npm -v
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Node Version Management - nvm installieren

```
# Installationsscript herunterladen und ausführen
# https://github.com/nvm-sh/nvm
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh |
bash
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Shell-Integration prüfen

```
# Prüfen, ob nvm in der Shell verfügbar ist
command -v nvm

# Nach Erfolg die installierte nvm-Version anzeigen
nvm --version
```

2026 Michael Reichart, copyleft CC BY-NC-SA

nvm im Alltag

- mehrere Node-Versionen parallel
- pro Shell oder Projekt wechseln
- "use" setzt nur die aktuelle Shell
- "default" gilt für neue Terminals

- Wenn Projekte verschiedene Laufzeiten brauchen, wird das sofort praktikabel.
- Alte und neue Node-Stände bleiben sauber getrennt.

2026 Michael Reichart, copyleft CC BY-NC-SA

Schneller Check

- nvm --version
- nvm ls
- Bei Fehler: Terminal neu öffnen
- Dann erst Versionen installieren und wechseln

2026 Michael Reichart, copyleft CC BY-NC-SA

npm private modules

Products Pricing Documentation Community

npm

Search packages

Search

Sign Up

Sign In

Build amazing

Pro Teams Pricing Documentation

npm

Search packages

Search

Popular libraries

- react
- lodash
- axios
- react-dom
- chalk
- commander
- tslib
- zod
- express
- inquirer

Discover packages

Front-end

Back-end

CLI

Documentation

CSS

Testing

IoT

Coverage

Mobile

Frameworks

Robotics

Math

By the numbers

Packages

4.469.608

Downloads · Last Week

226.855.025.638

Downloads · Last Month

879.859.881.111

2026 Michael Reichart, copyleft CC BY-NC-SA

Hallo Welt
4 Versionen

Der Praxisworkshop

2026 Michael Reichart, copyleft CC BY-NC-SA

Hallo Welt - Konsolenversion

```
$ node
> function hw () { console.log("hello world"); }
> hw()
hello world


(To exit, press ^C again or type .exit)
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Hallo Welt - Programmversion

```
// hello world.js
// - - - - - - - - -

function hw () { console.log("hello world"); }
hw();

// - - - - - - - - -

$ node hello-world
hello world
$
```

2026 Michael Reichart, copyleft CC BY-NC-SA

server.js

```
import http from 'http';     // aka. http = require('http')

let host = 'http://localhost',
    port = 3000,
    server = null;

server = http.createServer(function (request, response) {
    response.writeHead(200, {'Content-Type': 'text/html'});
    response.end('<h1>Hello world</h1>!');
});

server.listen(port);

console.log('a simple web service on ' + host + ':' + port);
```

2026 Michael Reichart, copyleft CC BY-NC-SA

server.js ausführen

```
$ node server
```

2026 Michael Reichart, copyleft CC BY-NC-SA

app.js (Express-Version)

```
// Modules
// const path = require('path');
// const express = require('express');
// const pageProcessor = require('./routes/pageProcessor');

// ES Modules
import path from 'path';
import express from 'express';
import pageRoute from './route.js';

const app = express();

let port = 3000,
    host = 'localhost';

app.use(express.static(path.resolve('public')));

app.set('view engine', 'ejs');
app.set('views', path.resolve('./'));

app.use('/', pageRoute);

app.listen(port, () => console.log(`Server running at http://${host}:${port}`));
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Projektstruktur

```
app.js
route.js
view.ejs

data.json

/static
  /assets
    /css
      style.css
    /js
      script.js
```

2026 Michael Reichart, copyleft CC BY-NC-SA

view.ejs

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width">
    <title>Table View</title>
    <link rel="stylesheet" href="assets/css/style.css">
</head>
<body>
    <h1>Page View</h1>
    <p><%= data %></p>
    <script src="assets/js/script.js"></script>
</body>
</html>
```

2026 Michael Reichart, copyleft CC BY-NC-SA

route.js

```
import express from 'express';
import path from 'path';
const router = express.Router();
import {
    readFile
} from 'fs/promises';
const data =
    JSON.parse(
        await readFile(
            new URL('./data.json',
                import.meta.url)
        )
    );
function onRequest(request, response) {
    response.render('view', {
        data: data.message
    });
}
router.get('/', onRequest);
export default router;
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Node Monitoring

2026 Michael Reichart, copyleft CC BY-NC-SA



nodemon reload, automatically.

Nodemon is a utility that will monitor for any changes in your source and automatically restart your server. Perfect for development. Install it using **npm**.

Just use `nodemon` instead of `node` to run your code, and now your process will automatically restart when your code changes. To install, get **node.js**, then from your terminal run:

```
npm install -g nodemon
```

Features

- Automatic restarting of application.
- Detects default file extension to monitor.
- Default support for node & coffeescript, but easy to run any executable (such as python, make, etc).

2026 Michael Reichart, copyleft CC BY-NC-SA

Nodemon global installieren

```
$ [sudo] npm install -g nodemon
Password:

. . .

+ nodemon@1.18.9
updated 1 package in 7.654s

. . .

npm config -g set proxy http://172.16.20.127:8080/
npm config -g set https-proxy http://172.16.20.127:8080/
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Auto restart mit nodemon

```
$ nodemon hello-world.js

$ nodemon --debug ./server.js 80
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Nodemon konfigurieren

```
{
  "name": "nodemon",
  "homepage": "http://nodemon.io",
  ...
  ... other standard package.json values
  ...
  "nodemonConfig": {
    "ignore": ["test/*", "docs/*"],
    "delay": "2500"
  }
}
```

2026 Michael Reichart, copyleft CC BY-NC-SA

About Nodemon

```
https://github.com/remy/nodemon
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Ein Node Projekt aufsetzen

2026 Michael Reichart, copyleft CC BY-NC-SA

Ein Nodeprojekt initialisieren

```
npm init

npm fragt alle Informationen für das Projekt ab und erzeugt dann
eine package.json.
```

2026 Michael Reichart, copyleft CC BY-NC-SA

package.json Beispiel

```
{
  "name"        : "my-project",
  "version"     : "1.0.0",
  "description" : "Lorem descriptum ipsum dolor sit.",
  "main"        : "./index.js",
  "directories" : {
                    "doc": "docs"
                  },

 "repository": {
    "type"   : "git",
    "url"    : "git+https://github.com/account/
                project.git"
  },

  "keywords" : [ "comma", "separated",
                 "value", "and", "word", "list" ],

  "author"   : "Michael Reichart",

  "license"  : "MIT",

  "bugs": {
    "url"    : "https://github.com/account/
               project/issues"
  },

  "homepage" : "https://github.com/account/
               project#readme",

  "type"      : "module",

   "scripts"  : {
      "watch" : "nodemon",
      "test"  : "echo \"Error: no tests\" && exit 1"
    },

  "dependencies" : {
     "ejs"     : "^3.1.7",
     "express" : "^4.18.1",
  },

   "devDependencies" : {
    "connect-livereload" : "^0.6.1",
  }
}
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Ein Nodeprojekt faul initialisieren

```
npm init -y
```

npm schreibt alle hinterlegten Informationen automatisch in eine package.json.

2026 Michael Reichart, copyleft CC BY-NC-SA

Ein Nodeprojekt initialisieren

```
npx license mit
uses the license package to download a license of choice, in this
case the MIT license

npx gitignore
node uses the gitignore package to automatically download the
relevant .gitignore file from GitHub’s repo

npx covgen
uses the covgen package to generate the Contributor Covenant and
give your project a code of conduct that will be welcoming to all
contributors
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Initiale npm package-Werte einstellen

```
npm config list | grep init

npm config set init.author.name "Your name"
npm config set init.author.email "your@email.com"
npm config set init.author.url "https://your-url.com"
npm config set init.license "MIT"
npm config set init.version "1.0.0"
```

Einmal hinterlegt, werden bei npm init -y die richtigen Werte eingesetzt.

2026 Michael Reichart, copyleft CC BY-NC-SA

Ausgaben über die Konsole

2026 Michael Reichart, copyleft CC BY-NC-SA

Ausführungszeit messen

```
console.time('my time measuring');

let value  = 'a primitive value',
    object = {
        prename  : 'Michael',
        lastname : 'Reichart'
    };

console.log(value);
console.dir(object);

console.timeEnd('my time measuring')
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Konsolenbefehle

```
console.assert(value[, message][, ...args])
console.dir(obj[, options])
console.table(obj)
console.error([data][, ...args])
console.info([data][, ...args])
console.log([data][, ...args])
console.time(label)
console.timeEnd(label)
console.trace(message[, ...args])
console.warn([data][, ...args])
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Debugging

2026 Michael Reichart, copyleft CC BY-NC-SA

Manual Debugging

2026 Michael Reichart, copyleft CC BY-NC-SA

```
$ node inspect hello-world.js

< Debugger listening on 127.0.0.1:5858
connecting to 127.0.0.1:5858 ... ok
break in hello-world.js:2
  1 // - - - - - - - - - - -
> 2 console.time('processed time');
  3 // - - - - - - - - - - -
  4
debug>
```

2026 Michael Reichart, copyleft CC BY-NC-SA

server.js

```
import http from 'http';

let host = 'http://localhost',
    port = 3000,
    server = null;

server = http.createServer(function (request, response) {
    response.writeHead(200, {'Content-Type': 'text/html'});
    debugger;
    response.end('<h1>Hello world</h1>!');
});

server.listen(port);

console.log('a simple web service on ' + host + ':' +
port);
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Stepping

```
cont, c - Continue execution
next, n - Step next
step, s - Step in
out,  o - Step out
pause   - Pause running code
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Breakpoints

```
setBreakpoint(),
sb()                        - Set breakpoint on current line

setBreakpoint(line),
sb(line)                    - Set breakpoint on specific line

setBreakpoint('fn()'),
sb(...)                       - Set breakpoint on a first statement
                            in functions body

setBreakpoint('script.js', 1),
sb(...)                       - Set breakpoint on first line of
script.js

clearBreakpoint('index.js', 15),
cb(...)                       - Clear breakpoint in index.js on line
15
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Debugging mit den Chrome Devtools

2026 Michael Reichart, copyleft CC BY-NC-SA

Der Nodejs V8 Inspector

- Die Chrome Extension "**nodejs V8 inspector**" in einer aktuellen Chromeversion installieren.
- startet einen Port 9229, über den die Chrome Devtools Fehlermeldungen anzeigen und der Code fehlerbehandelt werden kann.

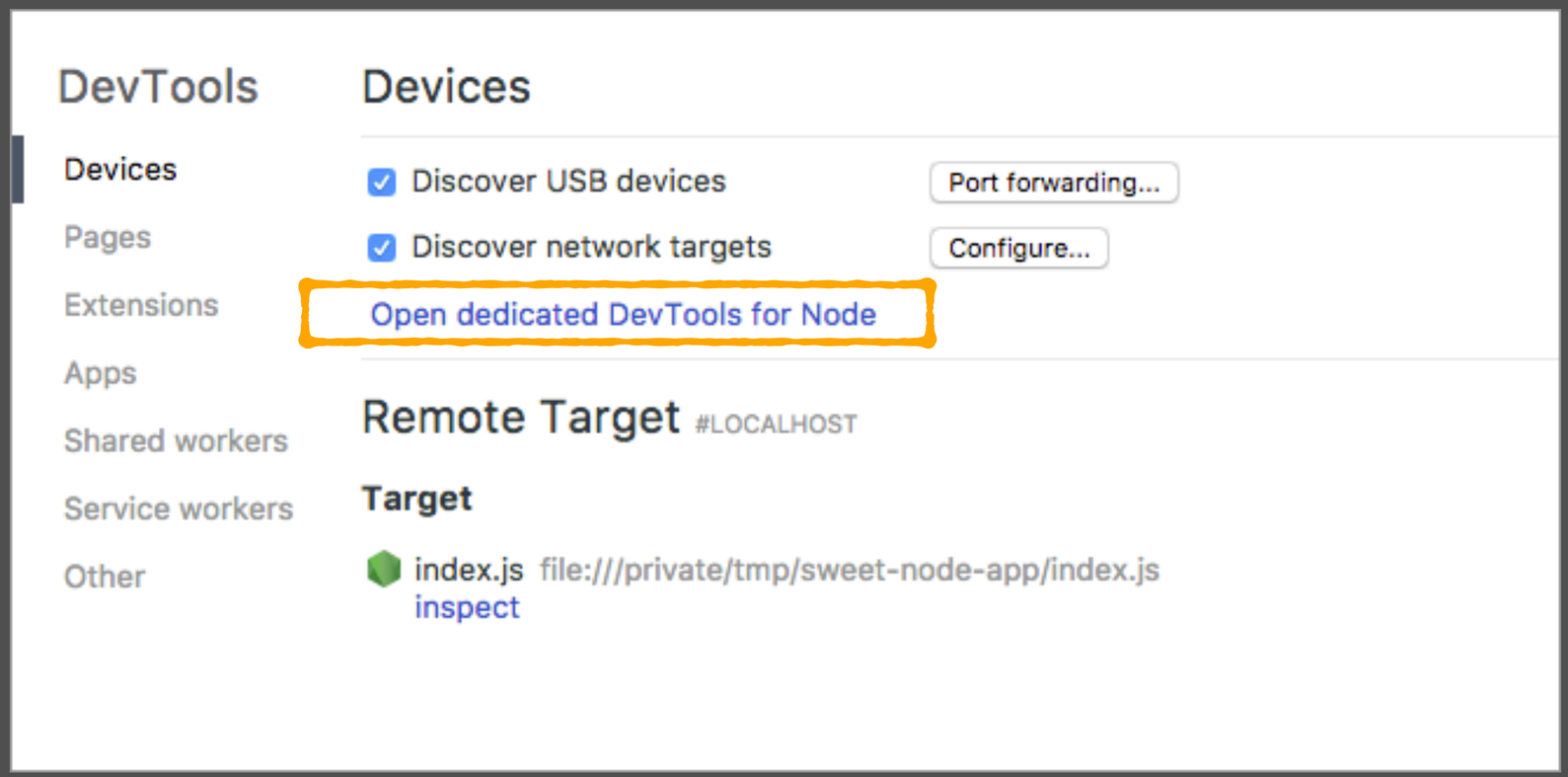
```
+ node --inspect index.js

  # Break on the first
  statement of the script
  with --inspect-brk

  node --inspect-brk
  index.js
```

2026 Michael Reichart, copyleft CC BY-NC-SA

- `chrome://inspect`



2026 Michael Reichart, copyleft CC BY-NC-SA

Software Architektur – Sichten und Schichten

Anwendungsarchitekturen (Softwarearchitektur)
OSI, 4+1 Sichten, Client-Server, 3- und N-Tier-Systeme

2026 Michael Reichart, copyleft CC BY-NC-SA

4+1 - Sichten-Modell

- **logical view** - Funktionalität des Systems für den Endnutzer. (Lastenheft-Team Fragen den Endnutzer)

- **process view** - sie verdeutlicht die Prozesse des Systems, sowie deren Laufzeitverhalten und Kommunikation. Die Prozessicht beschreibt Parallelität, Verteilung, Integration, Performance und Skalierbarkeit. (Requirements Engineer)

- **development view** (implementation view) - beschreibt das System vom Standpunkt eines Entwicklers. (20% Scrummaster als Teamkoordinator)

- **physical view** (deployment view) beschreibt das System vom Standpunkt des Systemarchitekten. Es beschäftigt sich mit der Verteilung der Softwarekomponenten auf physikalischer Ebene (Hardware) und der Kommunikation zwischen diesen Komponenten.

- **scenarios**: Die fünfte Sicht zeigt wichtige Anwendungsfälle oder Anwendungsszenarien. Sie beschreiben Abläufe zwischen Komponenten bzw. Prozessen. Sie helfen, Architekturelemente zu identifizieren, zu veranschaulichen und die Architektur zu überprüfen.
Sie dienen auch als Startpunkt für erste Architekturtests bzw. Implementierungsentwürfe.

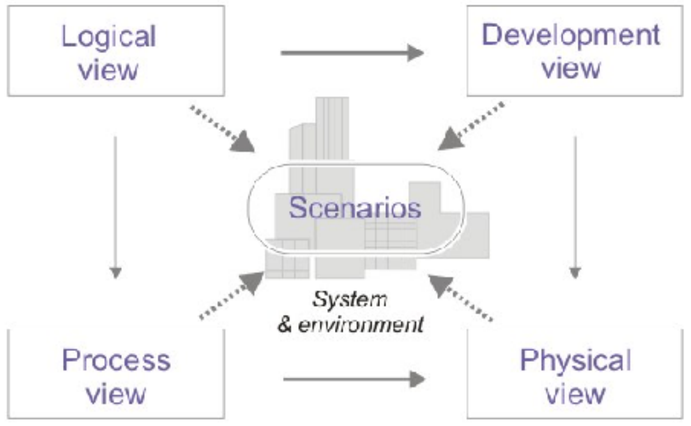
2026 Michael Reichart, copyleft CC BY-NC-SA

Sicht des Anwenders

UML-Diagrammformen:
Klassendiagramm,
Kommunikationsdiagramm,
Sequenzdiagramm
u.a.

Sicht des Entwicklers

UML-Diagrammformen:
Komponentendiagramm,
Paketdiagramm



Dynamik des Systems; Laufzeitverhalten

UML-Diagrammform:
Anwendungsfalldiagramm

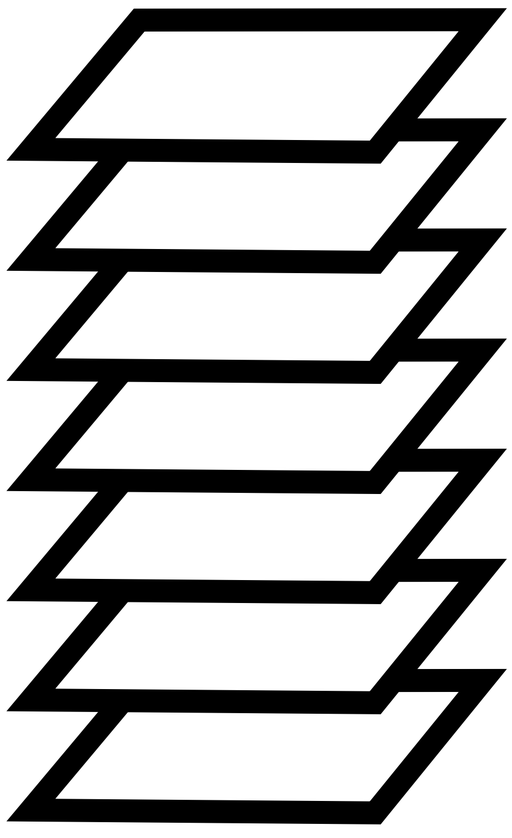
Sicht des Systemarchitekten

UML-Diagrammform:
Verteilungsdiagramm

© 2026 Michael Reichart, copyleft CC BY-NC-SA

OSI - Open Systems Interconnection

ws, **http**, ftp, dns, irc	7. application (end user layer)	data
ssl, **tls,** imap, mpeg, jpeg	6. presentation (syntax layer)	
API's, **sockets**, winsock	5. session (sync & send to port)	
tcp, udp	4. transport (end-to-end connections)	
ip, icmp, igmp, ipsec	3. network (packets)	
ppp, atm, **ethernet**, switch	2. data link (frames)	
coax, **wireless**, **gsm**, fiber, **usb**, bluetooth ...	1. physical	



Die sieben Ebenen des OSI Modells

2026 Michael Reichart, copyleft CC BY-NC-SA

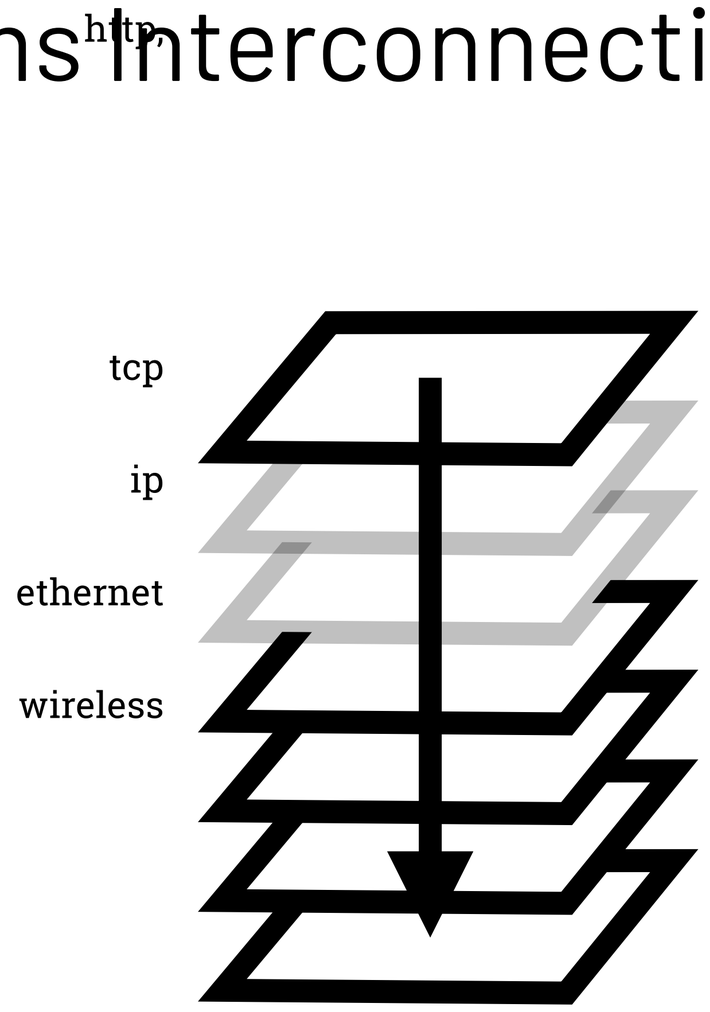
OSI - Open Systems Interconnection

If you use your web browser to navigate to **http://www.gfu.net**, this communication uses the following protocols from each layer, starting at layer 7:

HTTP → TCP → IP → Ethernet.

On the other hand, entering **https://www.gfu.net** would use

HTTP → SSL → TCP → IP → Ethernet.



7. application (end user layer)

6. presentation (syntax layer)

5. session (sync & send to port)

4. transport (end-to-end connections

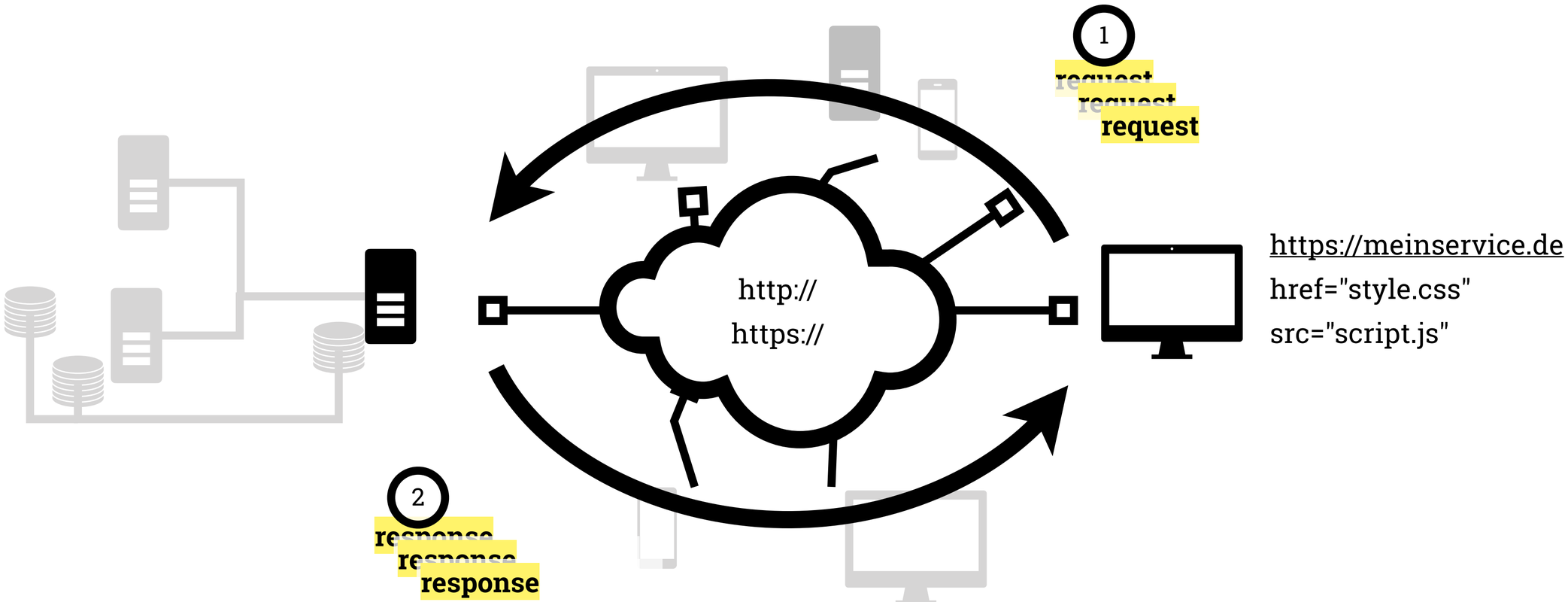
3. network (packets)

2. data link (frames)

1. physical

2026 Michael Reichart, copyleft CC BY-NC-SA

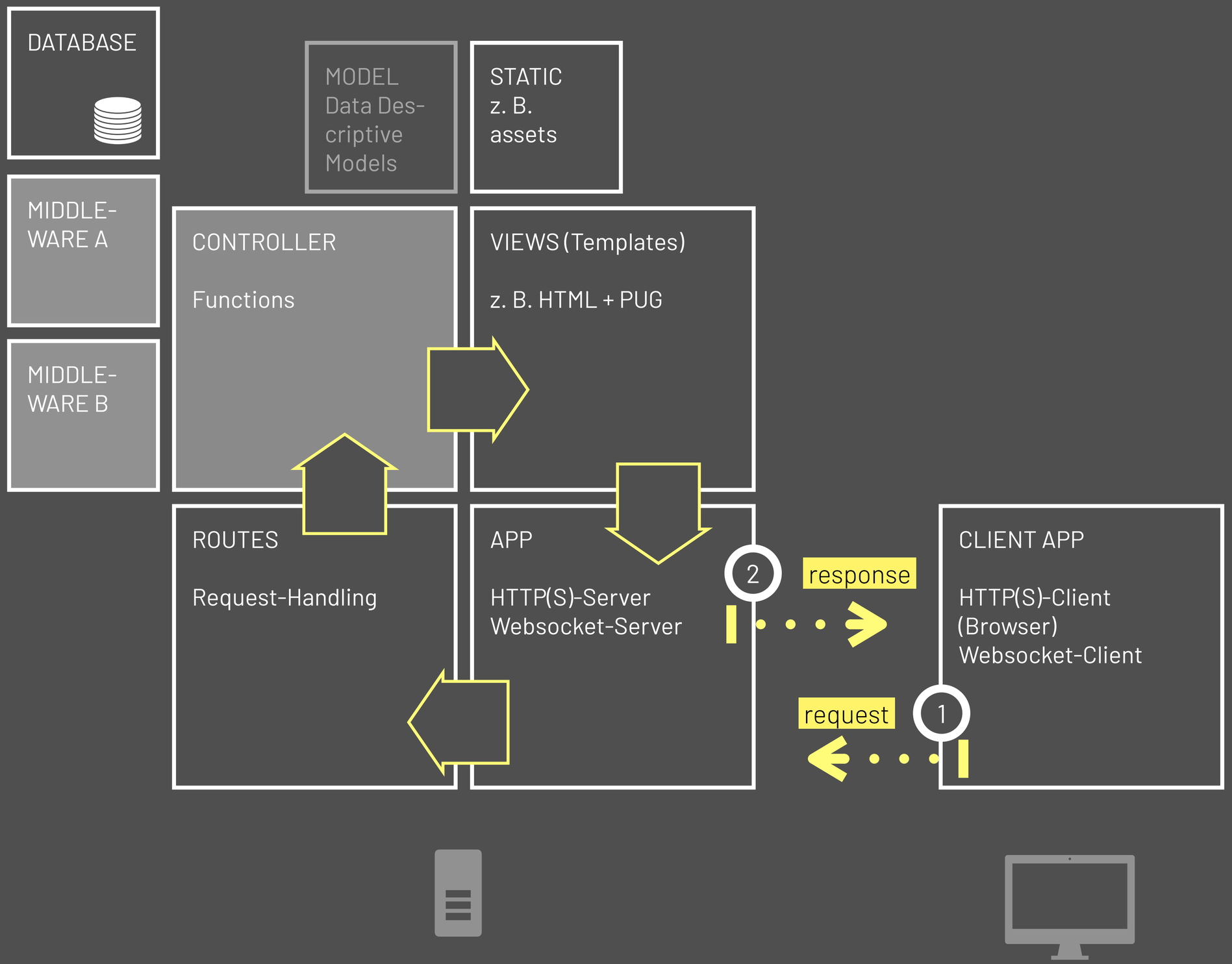
Server - Netzwerk - Client



Server

Client

2026 Michael Reichart, copyleft CC BY-NC-SA



2026 Michael Reichart, copyleft CC BY-NC-SA

Ein nodejs-Modul aufbauen

2026 Michael Reichart, copyleft CC BY-NC-SA

ECMAScript Module vs. CommonJS Modules

- EScript Module sind der offizielle Standard, um wiederverwendbaren Code zu schreiben.
- ES Modules verwenden `import` and `export` Statements.
- ES Module verwenden die Dateiendung `.js` oder `.mjs`.
- CommonJS Module folgen einer nodejs-Syntax. Jede nodejs-Datei bildet automatisch ein eigenes Modul.
- CommonJS Module verwenden `require()` und `exports` Statements.
- ES Module verwenden die Dateiendung `.js` oder `.cjs`.

2026 Michael Reichart, copyleft CC BY-NC-SA

Ein CommonJS Modul schreiben

```
// circle.cjs

let _area = null;

_area = function (radius) {
    return Math.PI * radius *
radius;
};

module.exports.area = _area;
```

```
// app.js

const circle =
      require('./circle.cjs');

let radius = 5;

console.log(circle.area(radius
));

> 78.53981633974483
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Ein ES Modul schreiben und importieren

```
// circle.mjs

const area = function(radius) {
   return
     Math.PI * radius
     * radius;
};

const circumference = function
(radius) { ... }

// Named export

export { area, circumference }
```

```
// app.js

import { area } from './
circle.mjs';

let radius = 5;

console.log( area(radius) );

> 78.53981633974483circle
```

2026 Michael Reichart, copyleft CC BY-NC-SA

```
// Benannter Export
function cube () { return value }
const foo = Math.sqrt(2);
export {cube, foo}

import {cube, foo} from …
```

```
// default exportiert nur eines je Script
// sheepController.js

export default class Sheep() {}

import Sheep from ‚sheepController.js'
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Module verwalten und installieren

2026 Michael Reichart, copyleft CC BY-NC-SA

Abhängigkeiten in der package.json

```
"dependencies" : {
  "express" : "^4.18.1",
 },

"devDependencies" : {
  "webpack"       : "*",
   "webpack-cli" : "^4.9.2"
}
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Ein Module installieren

```
$ npm install


$ npm install express
$ npm install --save-dev webpack webpack-cli
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Ein Modul einbinden

```
import http    from 'http';
import fs      from 'fs';
import express from 'express';

// let
//  http = require('http'),
//  fs = require(fs),
//  express = require('express');
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Versionierung der Module

2026 Michael Reichart, copyleft CC BY-NC-SA

major.minor.patch

```
"modulename" : " v1.23.3"

Auf Grundlage einer Versionsnummer von MAJOR.MINOR.PATCH
werden die einzelnen Elemente folgendermaßen erhöht:

MAJOR   wird erhöht, wenn API-inkompatible Änderungen
veröffentlicht werden.

MINOR   wird erhöht, wenn neue Funktionalitäten, welche
kompatibel zur bisherigen API sind, veröffentlicht werden.

PATCH   wird erhöht, wenn die Änderungen ausschließlich API-
kompatible Bugfixes umfassen.

Außerdem sind Bezeichner für Vorveröffentlichungen und
Build-Metadaten als Erweiterungen zum MAJOR.MINOR.PATCH
Format verfügbar.
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Modul-Versionierung

```
{
  "dependencies" : {
    "modulename" : " v1.23.3",
    "modulename" : "  1.23.3",

    "modulename" : " >1.23.1",
    "modulename" : ">=1.23.2",
    "modulename" : "<=1.23.3",
    "modulename" : " <1.23.4",

    "modulename" : " >1.23.1 <1.23.4",
    "modulename" : " 1.22.9 || >1.23.1 <1.23.4",

  }
}
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Modul-Versionierung

```
"dependencies" : {
  "modulename" : "1.23.1 - 1.23.5", // inclusive set
  "modulename" : "1.2.3",           // === 1.2.3

  "modulename" : "1.x",
  "modulename" : "1.2.*",           // 1.2.15    -> 1.2.11 ist möglich

  "modulename" : "~1",              // >= 1.0.0   < 2.0.0
  "modulename" : "~1.2",            // >= 1.2.0   < 1.3.0
  "modulename" : "~1.2.3",          // >= 1.2.3   < 1.3.0

  "modulename" : "^1.2.3",          // >= 1.2.3   < 2.0.0
  "modulename" : "^1.2.x",          // >= 1.2.0   < 2.0.0
}
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Wann wird die Versionsnummer angepasst?

- Die Version **1.0.0** definiert die öffentliche API. Ab dieser Veröffentlichung hängt die Art und Weise, wie die Versionsnummer erhöht und verändert wird, von den folgenden Regeln ab.

- Die Patch Version Z (x.y.**Z** | x > 0) muss (MUST) erhöht werden,
  -> wenn ausschließlich API-kompatible Bugfixes eingeführt werden.

- Die Minor Version Y (x.**Y**.z | x > 0) muss (MUST) erhöht werden,
  -> wenn neue Funktionalitäten, welche kompatibel zur bisherigen API sind, veröffentlicht werden.
  -> wenn eine Funktion der öffentlichen API als deprecated markiert wird.
  -> Die Patch Version muss (MUST) dann auf Null zurückgesetzt werden.

- Die Major Version X (**X**.y.z | X > 0) muss (MUST) immer dann erhöht werden,
  -> wenn API-inkompatible Änderungen in die öffentlichen API eingeführt werden.
  -> Wenn diese Versionsnummer erhöht wird, muss (MUST) sowohl die Minor Version als auch die Patch Version auf Null zurückgesetzt werden.

-

2026 Michael Reichart, copyleft CC BY-NC-SA

"https://semver.org/lang/de/"

Node Module folgen der Semantischen Versionierung

2026 Michael Reichart, copyleft CC BY-NC-SA

Native Bibliotheken

Utilities Klasse

2026 Michael Reichart, copyleft CC BY-NC-SA

```
util.debuglog(section)
util.deprecate(function, string)
util.format(format[, ...args])
util.inherits(constructor, superConstructor)
util.inspect(object[, options])
```

2026 Michael Reichart, copyleft CC BY-NC-SA

```
%s - String.
%d - Number (both integer and float).
%j - JSON. Replaced with the string '[Circular]' if
the argument contains circular references.
%% - single percent sign ('%'). This does not
consume an argument.

util.format('%s:%s', 'foo', 'bar', 'baz'); //
'foo:bar baz'
```

2026 Michael Reichart, copyleft CC BY-NC-SA

```
dataString = util.format(
    '%s-%s-%s %s:%s:%s Lorem ipsum dolor sit.\n',
     now.getFullYear(),
    (now.getMonth() + 1),
     now.getDate(),
     now.getHours(),
     now.getMinutes(),
     now.getSeconds()
    );
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Ereignisse verarbeiten

2026 Michael Reichart, copyleft CC BY-NC-SA

Eventhandler und -listener

- Viele Objekte senden in node Events aus: ein `net.server` sendet einen Event, wenn sich ein Client verbindet. Ein `fs.readStream` sendet ein Event, wenn die Datei geöffnet ist.

- Jedes Objekt, das Events sendet ist eine Instance des **events.EventEmitter Konstruktors**. Diese kann über `require("events");` eingebunden werden.

- Funktionen können an Objekte gebunden werden, wenn diese einen Event gesendet haben: Diese Funktionen werden Eventhandler genannt.

- In Eventhandlern verweist `this` auf das Event - aussendende Objekt.

2026 Michael Reichart, copyleft CC BY-NC-SA

Eventlistener setzen

```
emitter.addListener(event, listener)
emitter.on(event, listener)
```

```
server.on('connection', function (stream) {
        console.log('someone connected!');
});
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Einmal - Event

```
emitter.once(event, listener)
```

```
server.once('connection', function (stream) {
        console.log('Oh, a first user!');
});
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Löschen von Eventlistenern

```
emitter.removeListener(event, listener)
```

```
var callback = function(stream) {
 console.log('someone connected!');
};
```

```
server.on('connection', callback);
// ...
server.removeListener('connection', callback);
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Weiteres ...

```
emitter.removeAllListeners([event])


emitter.setMaxListeners(n) // 0 == unbegrenzt
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Dateisystem

2026 Michael Reichart, copyleft CC BY-NC-SA

readFile

```
fs.readFile(filename, function(error, content) {
  if (error) {
    throw error;
  };
  console.log(content.toString());
};);
```

2026 Michael Reichart, copyleft CC BY-NC-SA

writeFile

```
function() {
    // a - append, ggf. new
    fs.open(logFile, 'a', function(error, handle) {
      if (error) {
        throw error;
      };

      let
        logFileString = null,
        buffer = null;

      logFileString = (i++) + ': Ipsum Lorem Nullam Egestas\n';

      buffer = new Buffer(logFileString);

      fs.write(handle, buffer, 0, buffer.length, null, function() {
        fs.close(handle, function() { /* idle */ });
      });

    });
  };
```

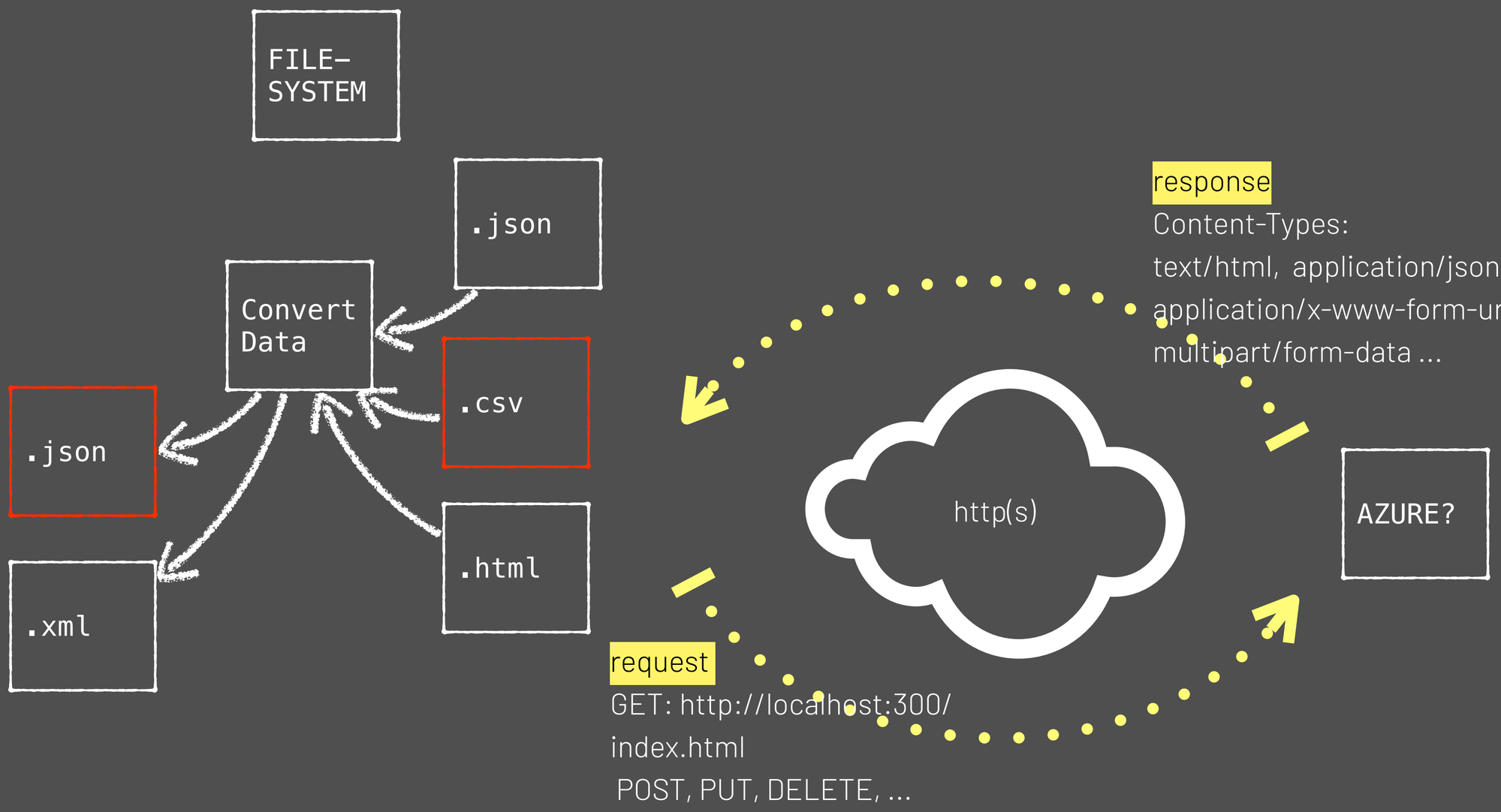
2026 Michael Reichart, copyleft CC BY-NC-SA

Watchfile

```
fs.watchFile(logFile, function(currentTime,
previousTime) {
   // atime - access time
    // mtime - modify time
    // ctime - current time (?)
    console.log('current time is ' +
currentTime.mtime);
    console.log('previous time is ' +
previousTime.mtime);

 });
```

2026 Michael Reichart, copyleft CC BY-NC-SA



Anwendungsfall Filesystem

2026 Michael Reichart, copyleft CC BY-NC-SA

Buffer

2026 Michael Reichart, copyleft CC BY-NC-SA

Buffer (content to Filesystem to content)

```
// read a file
onReadFile = function (error, buffer) {
    if (error) {
        throw error;
    }
    console.log(buffer.toString());
}
```

```
// some content
string = getDateString() + ' Lorem ipsum ...\n';

// string to buffer
buffer = Buffer.from(string);
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Essentielle Bibliotheken

Express

2026 Michael Reichart, copyleft CC BY-NC-SA

Express Basis Applikation

```
import express from 'express';
var express = require('express');

const app = express();

app.get('/', function(request, response){
  response.send('hello world');
});

app.listen(3000);
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Die ersten App-Methoden

```
app.get, app.post, app.put, ...,
app.all,
app.param     - Routing HTTP requests.
app.route     - Configuring middleware.
app.render    - Rendering HTML views.
app.engine    - Registering a template engine.
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Routes

```
app.get('/', callback);

Die Pfadangabe ('/') kann
- ein Stringausdruck,
- ein Pfadmuster,
- ein regulärer Ausdruck,
- oder ein Array mit einer Kombination

daraus sein.
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Das Request Objekt

- bildet den HTTP Request ab.
- Es enthält Eigenschaften wie den Querystring, übergebene Parameter, den Request-Body, die HTTP-Headers und so weiter.
- http://expressjs.com/de/4x/api.html#req

2026 Michael Reichart, copyleft CC BY-NC-SA

Das Request Objekt

```
app.get('/user/:id', function(request, response) {
  res.send('user ' + request.params.id);
});

console.log('The views directory is ' +
request.app.get('views'));

console.log(request.baseUrl);

console.log(request.cookies.name);

console.log(request.hostname);
console.log(request.ip);
console.log(request.method);
console.log(request.path);
console.log(request.query);
console.log(request.route);
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Request Body

```
var app        = require('express')();
var bodyParser = require('body-parser');
var multer     = require('multer'); // v1.0.5
var upload     = multer(); // for parsing
                           multipart/form-data

// for parsing application/json
app.use(bodyParser.json());

// for parsing application/x-www-form-urlencoded
app.use(bodyParser.urlencoded({ extended: true }));

app.post('/profile', upload.array(), function (request,
response, next) {
  console.log(request.body);
  response.json(request.body);
});
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Das Response Objekt

- Bildet die HTTP Antwort, die Express bei einer Anfrage formuliert und abschickt.
- Es besitzt ausser einigen Eigenschaften vor allem die Methode, um eine Antwort zu bilden.

2026 Michael Reichart, copyleft CC BY-NC-SA

Das Response Objekt

```
response.append('Link', ['<http://localhost/>', '<http://
localhost:3000/>']);
response.append('Set-Cookie', 'foo=bar; Path=/; HttpOnly');
response.append('Warning', '199 Miscellaneous warning');

response.attachment('path/to/logo.png');

response.cookie('name', 'tobi', { domain: '.example.com', path: '/
admin', secure: true });

response.download('/report-12345.pdf');

res.send(new Buffer('whoop'));
res.send({ some: 'json' });
res.send('<p>some html</p>');
res.status(404).send('Sorry, we cannot find that!');
res.status(500).send({ error: 'something blew up' });

response.end();
response.status(404).end();
```

2026 Michael Reichart, copyleft CC BY-NC-SA

```
response.json(null);
response.json({ user: 'tobi' });
response.status(500).json({ error: 'message' });

response.jsonp(null);
response.jsonp({ user: 'tobi' });
response.status(500).jsonp({ error: 'message' });
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Response Formate

```
res.format({
  'text/plain': function(){
    res.send('hey');
  },

  'text/html': function(){
    res.send('<p>hey</p>');
  },

  'application/json': function(){
    res.send({ message: 'hey' });
  },

  'default': function() {
    // log the request and respond with 406
    res.status(406).send('Not Acceptable');
  }
});
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Live Reload

2026 Michael Reichart, copyleft CC BY-NC-SA

Live Reload im Browser

```
npm install connect-livereload --save-dev
npm install livereload --save-dev

// - - - - - - - - - -

import connectLiveReload from 'connect-livereload';
import liveReload from 'livereload';

const liveReloadServer = liveReload.createServer();
liveReloadServer.server.once('connection', () => {
    setTimeout(() => {
        liveReloadServer.refresh("/")
    }), 100
});


...


// use with express
app.use(connectLiveReload({
    port: 35729
}));
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Crypto

2026 Michael Reichart, copyleft CC BY-NC-SA

salt, hash

```
// To generate the password (pseudo)
// PBKDF2 (Password-Based Key Derivation Function 2)
// - - - - - - - - - -
function hashPassword(password) {
    var salt       = crypto.randomBytes(128).toString('base64');
    var iterations = 10000;
    var hash       = pbkdf2(password, salt, iterations);

    return {
        salt: salt,
        hash: hash,
        iterations: iterations
    };
}

// To validate password (pseudo)
// - - - - - - - - - -
function isPasswordCorrect( savedHash, savedSalt,
                            savedIterations, passwordAttempt) {

    return savedHash == pbkdf2(passwordAttempt, savedSalt,
                               savedIterations);

}
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Cipher / Decipher
Symmetric Encryption

```
let
    crypto    = require('crypto'),
    algorithm = 'aes-192-cbc',
    password  = 'Password used to generate key',
    salt      = 'salt', // "Secured Automated Lending Technology"
    key, iv, cipher, decipher, encrypted, decrypted;

key = crypto.scryptSync(password, salt, 24);
iv  = Buffer.alloc(16, 0);

// Cipher
cipher = crypto.createCipheriv(algorithm, key, iv);

cipher.on('readable', () => {
    let chunk;
    while (null !== (chunk = cipher.read())) {
        encrypted += chunk.toString('hex');
    }
});

cipher.on('end', () => {
    console.log(encrypted);
});

cipher.write('some clear text data');
cipher.write('Lorem ipsum dolor sit');

cipher.end();
```

2026 Michael Reichart, copyleft CC BY-NC-SA

CIpher / Decipher
Symmetric Encryption

```
// Decipher
decipher = crypto.createDecipheriv(algorithm, key, iv);

decipher.on('readable', () => {
    while (null !== (chunk = decipher.read())) {
        decrypted += chunk.toString('utf8');
    }
});

decipher.on('end', () => {
    console.log(decrypted);
});

decipher.write(encrypted, 'hex');
decipher.end();
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Cipher / Decipher - update und final

```
// Using the .update() and .final() methods:

// Cipher (verschlüsseln)
// - - - - - - - - - - -
cipher     = crypto.createCipheriv(algorithm, key, iv);
encrypted  = cipher.update('some clear text', 'utf8', 'hex');
encrypted += cipher.final('hex');

console.log(encrypted);

// Decipher (entschlüsseln)
// - - - - - - - - - - -
decipher   = crypto.createDecipheriv(algorithm, key, iv);
decrypted  = decipher.update(encrypted, 'hex', 'utf8');
decrypted += decipher.final('utf8');

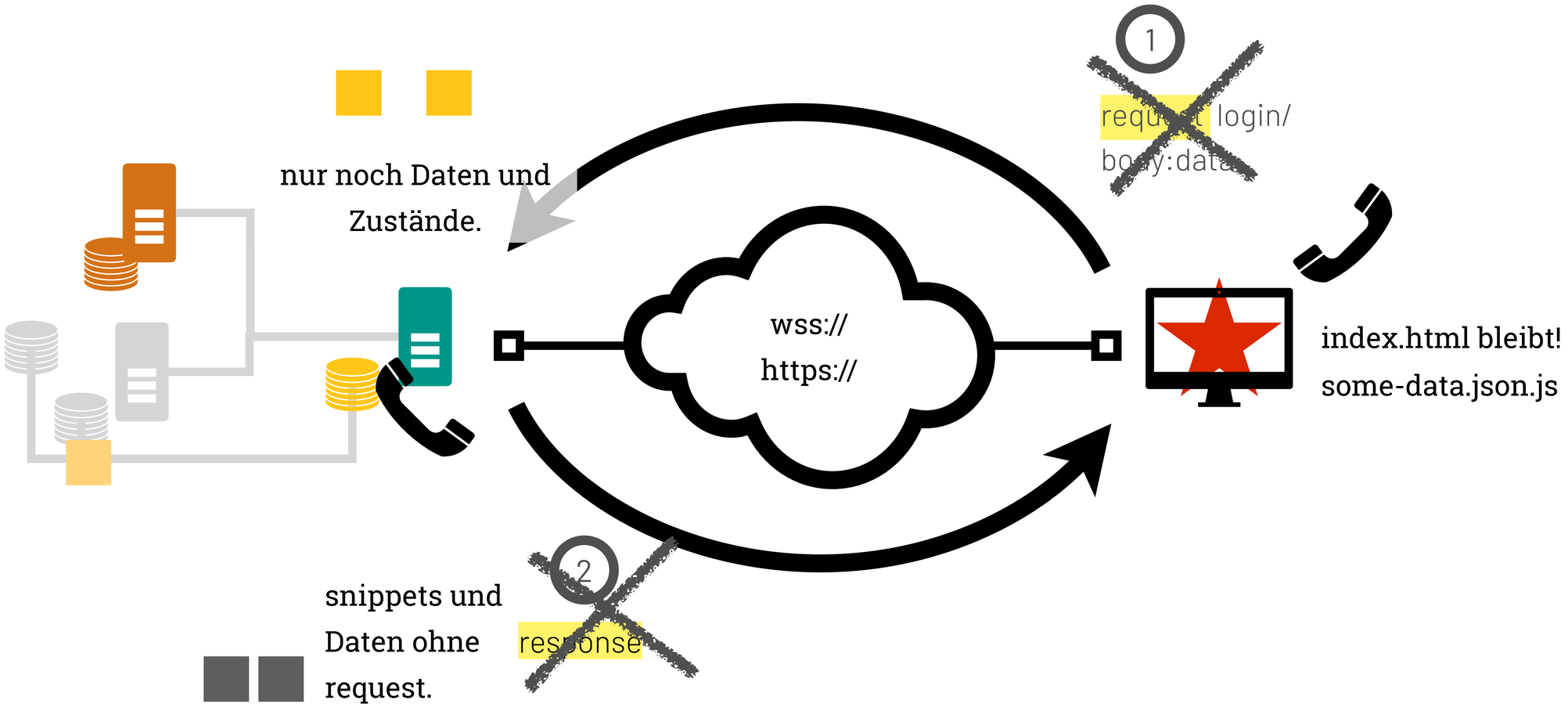
console.log(decrypted);
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Websockets

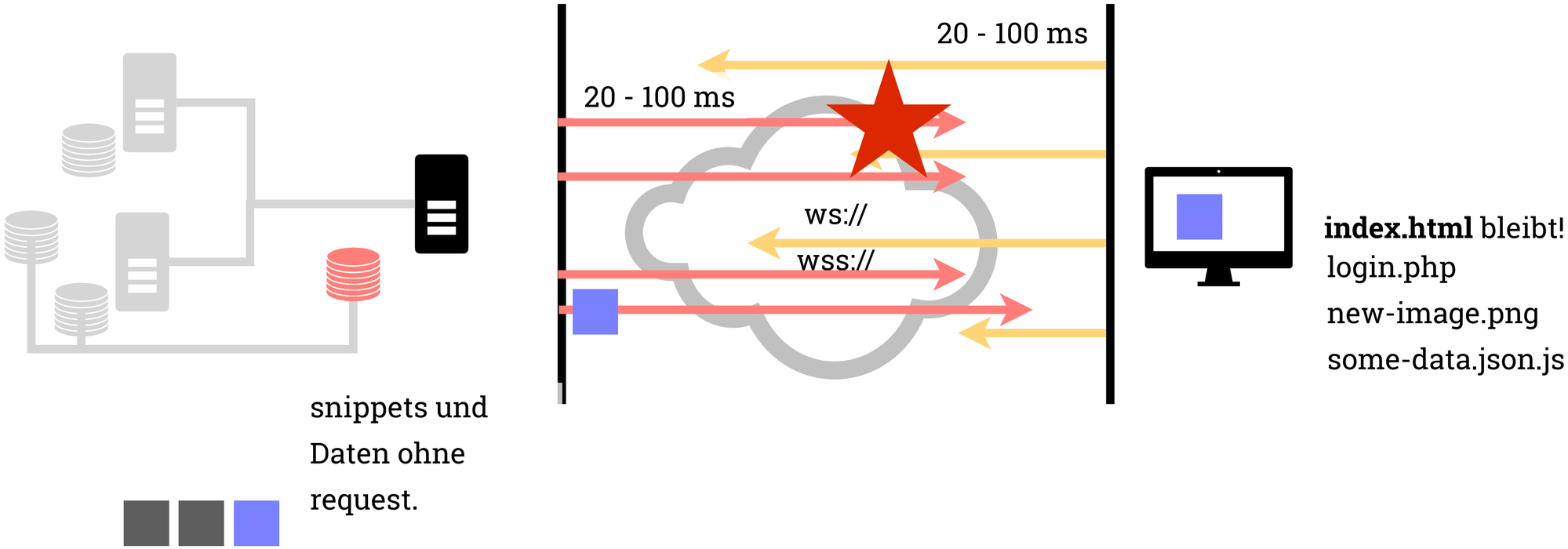
Einführung in Websockets

2026 Michael Reichart, copyleft CC BY-NC-SA



2026 Michael Reichart, copyleft CC BY-NC-SA

nur noch Daten und Zustände.



index.html bleibt!
login.php
new-image.png
some-data.json.js

snippets und Daten ohne request.

2026 Michael Reichart, copyleft CC BY-NC-SA

„Mit AJAX Requests kann ich über den Browser keine Modelleisenbahn steuern.“

–Jan Hickson

2026 Michael Reichart, copyleft CC BY-NC-SA

Nodejs Sockets

Javascript auf dem Server

2026 Michael Reichart, copyleft CC BY-NC-SA

Ein Socketproxy

```
// Websocket-Server (-> npm install ws!)

var WebSocketServer = require('ws').Server
var wss = new WebSocketServer({
    host: „192.168.2.1“,
    port: 8000
});

wss.on('connection', function(ws) {
    console.log('client verbunden...');

    ws.on('message', function(message)   {
        console.log('von Client empfangen: ' + message);
        ws.send('von Server empfangen: ' + message);
    });
});
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Der Browserpart zur Socketkommunikation

```
function connect() {
        // Websocket
        var socket = new WebSocket(„ws://192.168.2.1:8000“);

        socket.onopen = function() {
            console.log(„Socket Status: “
            + socket.readyState + „ (open)“);
        }

        socket.onmessage = function(msg) {
            console.log(„Empfangen: “ + msg.data);
        }

        socket.onerror = function (err) {
            console.log(„Ein Fehler ist aufgetreten.“);
        }

        socket.send(„Hallo Welt“);
}
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Die Javascript API für den Client

2026 Michael Reichart, copyleft CC BY-NC-SA

Das readyState Attribut enthält den Status der Verbindung

```
CONNECTING (numeric value 0)
Die Verbindung wurde noch nicht hergestellt.

OPEN (numeric value 1)
Die Verbindung steht, Kommunikation ist möglich.

CLOSING (numeric value 2)
Die Verbindung führt den Closing Handshake aus.

CLOSED (numeric value 3)
Die Verbindung wurde geschlossen oder konnte nicht
hergestellt werden.
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Die Websocket Methoden

```
mySocket = new WebSocket();

mySocket.onopen = function(evt) {
    console.log(„Connection open ...");
};

mySocket.onmessage = function(evt) {
    console.log( "Received Message: " + evt.data);
};

mySocket.onclose = function(evt) {
    console.log("Connection closed.");
};

mySocket.onerror = function(evt) {
    console.log("An error happened.");
};
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Socket öffnen und Daten senden

```
var mySocket = new WebSocket('ws://
game.example.com:12010/updates');

mySocket.onopen = function () {

    setInterval(function() {

    if (mySocket.bufferedAmount === 0) {
        mySocket.send( );
    }

  }, 50);

};
```

2026 Michael Reichart, copyleft CC BY-NC-SA

onmessage

```
mySocket.onmessage = function (event) {

    if (event.data === 'on') {
        turnLampOn();
    } else if (event.data === 'off') {
        turnLampOff();
    }

};
```

2026 Michael Reichart, copyleft CC BY-NC-SA

socket.io

Javascript auf dem Server

2026 Michael Reichart, copyleft CC BY-NC-SA

Die socket.io package.json

```
{
    "name": "socket-chat-example",
    "version": "0.0.1",
    "description": "my first socket.io app",
    "dependencies": {
        "express": "^4.18.1",
        "socket.io": "^4.5.1"
    }
}
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Chat-Beispiel - import und Weberver

```
const express = require('express');
const app = express();
const http = require('http');
const server = http.createServer(app);
const {
    Server
} = require("socket.io");

const io = new Server(server);

app.get('/', (req, res) => {
    res.sendFile(__dirname + '/index.html');
});
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Chat-Beispiel: Message-Handling

```
io.on('connection', (socket) => {
    console.log('a user connected');

    socket.on('disconnect', () => {
        console.log('user disconnected');
    });

    socket.on('chat message', (msg) => {
        io.emit('chat message', msg);
    });
});

server.listen(3000, () => {
    console.log('listening on *:3000');
});
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Chat: HTML-Scaffoldimg

```
<!DOCTYPE html>
<html>

<head>
    <title>Socket.IO chat</title>
    <style> ... </style>
</head>

<body>
    <ul id="messages"></ul>
    <form id="form" action="">
        <input id="input" autocomplete="off" /><button>Send</button>
    </form>

    <script src="/socket.io/socket.io.js"></script>
    <script>
        var socket = io();
        ...
   </script>
</body>

</html>
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Chat-Stylesheet

```
body {
    margin: 0;
    padding-bottom: 3rem;
    font-family: sans-serif;
}

#form {
    background: rgba(0, 0, 0, 0.15);
    padding: 0.25rem;
    position: fixed;
    bottom: 0;
    left: 0;
    right: 0;
    display: flex;
    height: 3rem;
    box-sizing: border-box;
    backdrop-filter: blur(10px);
}

#input {
    border: none;
    padding: 0 1rem;
    flex-grow: 1;
    border-radius: 2rem;
    margin: 0.25rem;
}

#input:focus {
    outline: none;
}
#form>button {
    background: #333;
    border: none;
    padding: 0 1rem;
    margin: 0.25rem;
    border-radius: 3px;
    outline: none;
    color: #fff;
}

#messages {
    list-style-type: none;
    margin: 0;
    padding: 0;
}

#messages>li {
    padding: 0.5rem 1rem;
}

#messages>li:nth-child(odd) {
    background: #efefef;
}
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Chat-Script: Form und Message-Handling

```
var form = document.getElementById('form');
var input = document.getElementById('input');

form.addEventListener('submit', function (e) {
    e.preventDefault();
    if (input.value) {
        socket.emit('chat message', input.value);
        input.value = '';
    }
});

socket.on('chat message', function (msg) {
    var item = document.createElement('li');
    item.textContent = msg;
    messages.appendChild(item);
    window.scrollTo(0, document.body.scrollHeight);
});
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Daten anbinden

ReST API

2026 Michael Reichart, copyleft CC BY-NC-SA

Was ist ReST?

- ReST ist ein Akronym für *Representational State Transfer*.
- Es basiert auf Web-Standards und dem HTTP-Protokoll
- Die ReST Architektur beschreibt sechs Vorschriften (nach Roy Fielding)

1. Uniform Interface (Einheitliche Schnittstelle)
2. Stateless (Zustandslosigkeit)
3. Cacheable (speicherbar, wiederverwendbar)
4. Client-Server-Modell
5. Layered System (mehrschichtig, hierarchisch)
6. Code on Demand (optional)

2026 Michael Reichart, copyleft CC BY-NC-SA

Client-Server-Modell

- REST verlangt ein Client-Server-Modell, will also das Nutzerinterface von der Datenhaltung getrennt sehen.
- Damit lassen sich Clients einfach auf verschiedene Plattformen portieren.
- (U. a. so) vereinfachte ServerKomponenten lassen sich gut skalieren.

2026 Michael Reichart, copyleft CC BY-NC-SA

Zustandslosigkeit

- Client und Server müssen zustandslos („stateless“) miteinander kommunizieren.
- Jede Anfrage eines Clients beinhaltet alle Informationen, die ein Server benötigt.
- Server können auf keinen gespeicherten Kontext zurückgreifen.
- Diese Einschränkung verbessert Visibilität, Zuverlässigkeit und Skalierbarkeit.
- Dafür nimmt REST Nachteile bei der Netzwerkperformanz in Kauf.
- Und Server haben keine Kontrolle über ein konsistentes Verhalten der Client-App.

2026 Michael Reichart, copyleft CC BY-NC-SA

Caching

+ Um die Netzwerkeffizienz zu verbessern, können Clients vom Server gesendete Antworten auch speichern und bei gleichartigen Requests später erneut verwenden.

+ Die Informationen müssen dem entsprechend als „cacheable“ oder „non-cacheable“ gekennzeichnet werden.

+ Die Vorteile responsiverer Anwendungen mit höherer Effizienz und Skalierbarkeit werden so mit dem Risiko erkauft, dass Clients auf veraltete Daten aus dem Cache zurückgreifen.

2026 Michael Reichart, copyleft CC BY-NC-SA

Layered System

- REST setzt auf mehrschichtige, hierarchische Systeme („*layered system*“).
- Jede Komponente kann ausschließlich jeweils direkt angrenzende Schichten sehen. Somit lassen sich beispielsweise Legacy-Anwendungen kapseln.
- Als Load Balancer agierende Vermittler („Intermediaries“) können die Skalierbarkeit verbessern.
- Als Nachteile dieses Constraints gelten ein zusätzlicher Overhead und erhöhte Latenzen.

2026 Michael Reichart, copyleft CC BY-NC-SA

Code-On-Demand

- Dieses Constraint fordert, dass die Funktionen von Clients über nachlad- und ausführbare Programmteile erweitert werden können – etwa in Form von Applets oder Skripten.
- Als optionales Constraint kann diese Bedingung in bestimmten Kontexten jedoch deaktiviert sein.

2026 Michael Reichart, copyleft CC BY-NC-SA

- ReST-Anwendungen verwenden HTTP Requests, um die vier CRUD Operationen auszuführen.

- C: create, R: read, U: update, und D: delete).

- GET - fordert Daten vom Server an

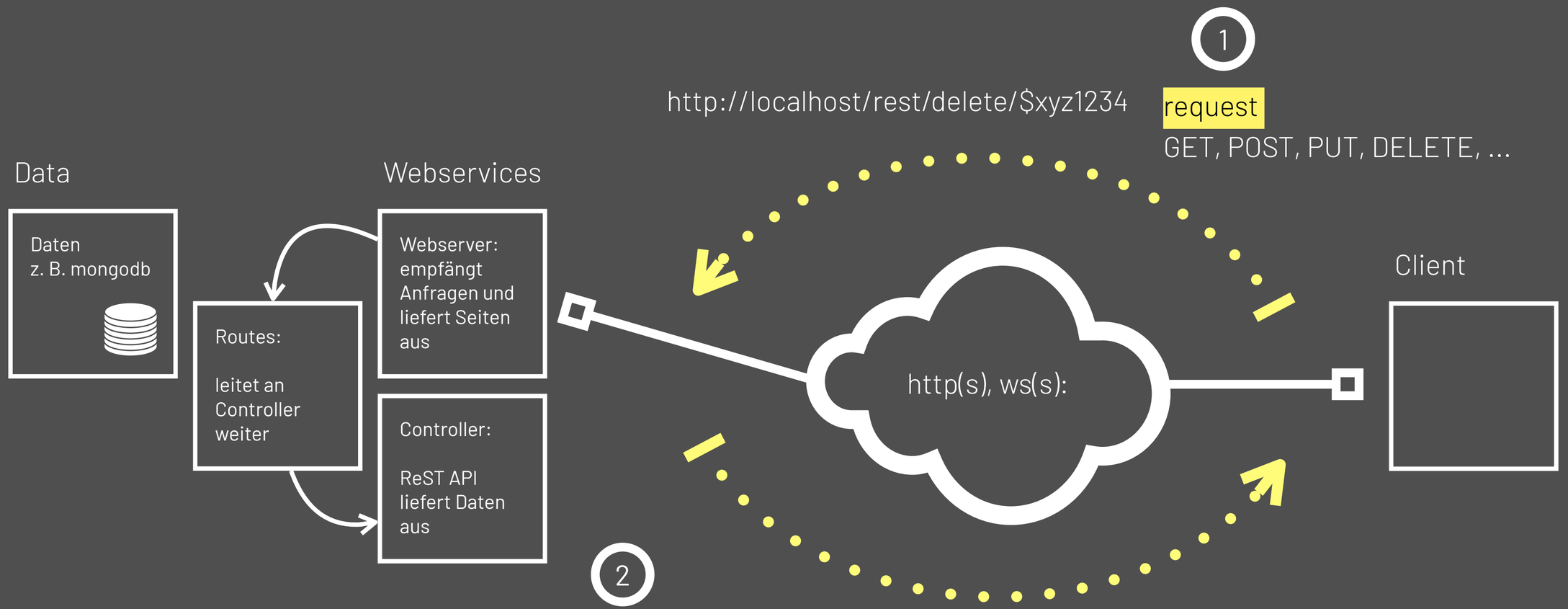
- POST - übermittelt Daten an den Server

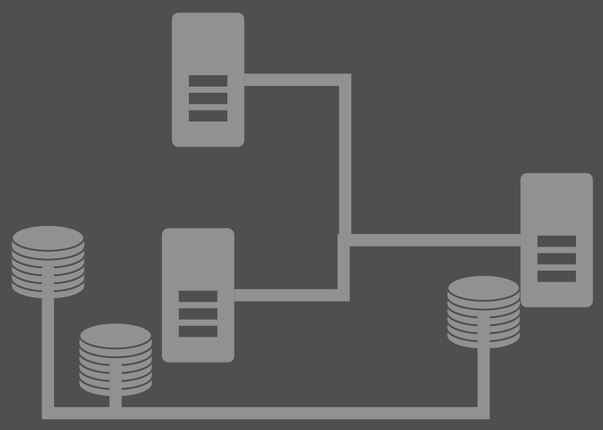
- PUT/PATCH - ändert bestehende Daten auf dem Server

- DELETE - löscht bestehende Daten auf dem Server

- ReST wird aus Methoden zusammengesetzt:
  base URL, URL, media types, etc.

2026 Michael Reichart, copyleft CC BY-NC-SA





Server - Web - Client



2026 Michael Reichart, copyleft CC BY-NC-SA

```
// A client request:
// http://localhost/rest/delete/$xyz1234
const response = await fetch(`rest/delete/${id}`, {
    method: 'DELETE',
    cache: 'no-cache',
    headers: {
              'Content-Type': 'application/json'
             },
});


// The server acts:
app.js:    app.use('/rest', restRoute);

restRoute: router.delete('/delete/:id?', deletePost);

restController:
let deletePost = (request, response, next) => {
    MongoClient.connect(url, (error, dbo) => {
        if (error) throw error;
        let database = dbo.db("blog"),
            myquery = { _id: new ObjectId(request.params.id) };

        database.collection("posts").deleteOne(myquery, (error, resultset) => {
            if (error) throw error;
            dbo.close();
        });
});
response.status(200).json({ "done": true });
```

2026 Michael Reichart, copyleft CC BY-NC-SA

https://www.ibm.com/de-de/cloud/learn/rest-apis

2026 Michael Reichart, copyleft CC BY-NC-SA

https://jinalshahblog.wordpress.com/2016/10/06/rest-api-using-node-js-and-mysql/

2026 Michael Reichart, copyleft CC BY-NC-SA

MySQL

2026 Michael Reichart, copyleft CC BY-NC-SA

A Database Connection

```
npm install mysql;

'use strict';
  // - - - - - - - - - - - -
  const mysql = require('mysql');
  const dbConfig = require('./db-config');

  let
    db = mysql.createConnection(dbConfig),
    sql = null;

  db.on('error', function(error) {
    console.log(error);
  });

  ...

  db.end();
```

2026 Michael Reichart, copyleft CC BY-NC-SA

The Database Connection Data

```
// db-config.json
{
  "host":       "localhost",
  "user":       "root",
  "password":   "root",
  "database" : "application",
  "port":       3206,

  "more": {
    "table": "user"
  }
}
```

2026 Michael Reichart, copyleft CC BY-NC-SA

SQL Queries

```
db.query('CREATE DATABASE IF NOT EXISTS application;');
db.query('USE application;');

// Tabelle anlegen
db.query('DROP TABLE IF EXISTS user;');

sql = "CREATE TABLE user ( " +
  "userId INT(11) AUTO_INCREMENT, " +
  "username VARCHAR(50), " +
  "email VARCHAR(50), " +
  "password VARCHAR(50), " +
  "PRIMARY KEY (userId) );";
db.query(sql);

sql = "INSERT INTO user " +
  "(username, email, password) " +
  "VALUES " +
  "('Michael', 'michael@zenbox.de', 'geheim')," +
  "('Paula', 'paula@zenbox.de', 'geheim')," +
  "('Klaus', 'klaus@zenbox.de', 'geheim');";

db.query(sql, function() {
  console.log('Datensätze geschrieben.');
});
```

2026 Michael Reichart, copyleft CC BY-NC-SA

MySQL in an index route 1
Connecting the database

```
'use strict';
 // - - - - - - - - - - - - -
 const express = require('express');
 const router = express.Router();
 const mysql = require('mysql');

 let
   dbConfig = require('../db-config.json'),
   db = null,
   query = null;

// connect the database service
dbConfig.database = dbConfig.more.database;
db = mysql.createConnection(dbConfig);

...

module.exports = router;
```

2026 Michael Reichart, copyleft CC BY-NC-SA

MySQL in an index route 2
A get request Query

```
router.get('/', function (request, response) {
  query = "SELECT * FROM user WHERE 1;";
  db.query(query, function (error, result) {
    if (error) { console.dir(error); process.exit(0); }
    // console.dir(result);

    response.render(
      'index',
      {
        title : ' a mysql result',
        result : result
      }
    );

  });
});
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Output Result as Table

```
<table>
  <thead>
    <tr>
      <% for (let key in result[0]) { %>
        <th>
          <%= key %>
        </th>
        <% } %>
    </tr>
  </thead>
  <tbody>
     <% for (let i=0, len=result.length; i<len; i+=1) { %>
        <tr>
          <% for (let key in result[i]) { %>
            <td>
              <%= result[i][key] %>
           </td>
            <% } %>
        </tr>
        <% } %>
  </tbody>
</table>
```

2026 Michael Reichart, copyleft CC BY-NC-SA

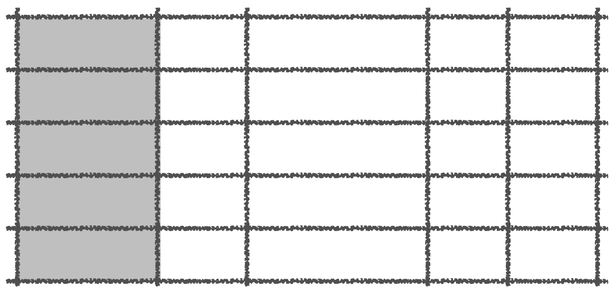
Mongodb

2026 Michael Reichart, copyleft CC BY-NC-SA

SQL vs. Not only SQL - noSQL

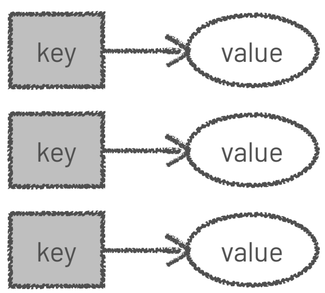
SQL

Relational

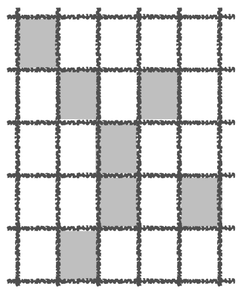


noSQL

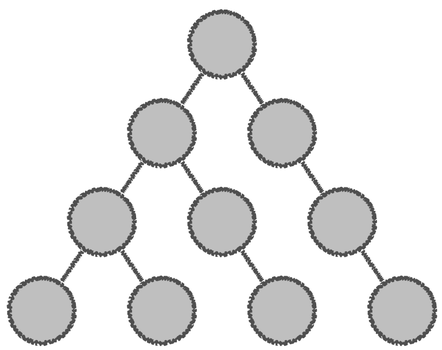
key-value
(i.e. localStorage)



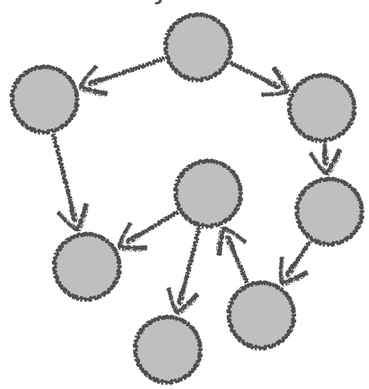
wide-column,
i.e. Azure tables



document store,
i. e. Mongo



graph,
i.e. Neo4j



2026 Michael Reichart, copyleft CC BY-NC-SA

Dokumentenorientierte Datenbanken

+ speichern Informationen in einfachen Tabellenstrukturen und Dokumenten.

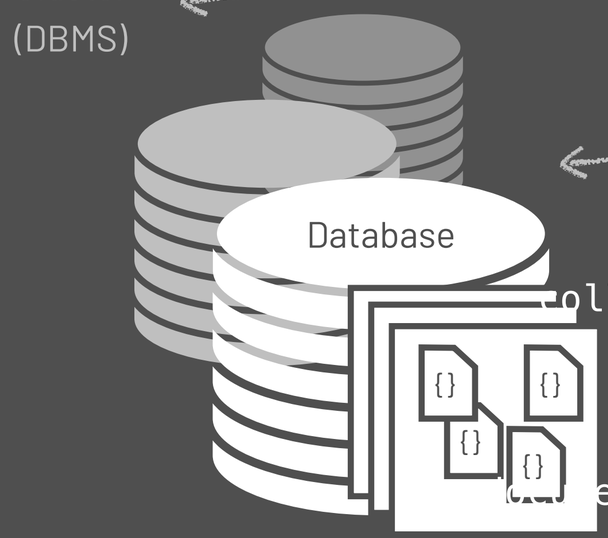
+ heissen auch *document stores*.

+ *document stores* sind nicht relational.

+ Die Daten sind teilstrukturiert. Sie folgen keiner festen Struktur, sondern tragen die Struktur in sich selbst.

+ Geordnet wird an Hand von Markierungen.

2026 Michael Reichart, copyleft CC BY-NC-SA




```
client = new MongoClient(uri);
client.connect();

database = client.db("my_db");

collection = database.collection("autor");
collection.insertOne({ ... });

document  = collection.findOne(query, options);
documents = collection.findMany( ... );

document = collection.find( ... )
                     .sort( ... )
                    .limit( ... );
```

Mongo-DB Begriffe

2026 Michael Reichart, copyleft CC BY-NC-SA

Collections in document stores

Collection: »Einträge«

```
{["id": 1,
          "autor": "Michael",
 "datum": "2015-03-23T11:19:21.000Z",
  "ueberschrift": "Mein neuer Blog",
"eintrag": "Hier der Text des Blogs…",
 "markierung": ["Allgemein", "News"],
           "kommentare": [{
            "autor": "klaus1",
  "datum": "2015-03-24T11:23:15.000Z",
      "kommentar": "Super Blog!"
                 },{
          "autor": "tom15",
"datum": "2015-03-25T11:29:15.000Z",
   "kommentar": "Unbedingt lesen!"
                }],

           [["id": 2,
         "autor": "Oliver",
                ...]
                }
```

Collection: »Benutzer«

```
{"benutzer": "Michael",
          "name": {
       "vorname": "Michael",
       "nachname": "Schwarze"
             },
 "password": "$5$6&7665!!223/34%4",

   "benutzer": "tom15",
     "password": ...
"password": ...
     ...
   }
```

Die Daten sind abfrage-orientiert gespeichert, ggf. redundant und lassen keine Beziehungen (JOIN) zu.

2026 Michael Reichart, copyleft CC BY-NC-SA

nodejs package installieren

```
npm install mongodb
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Einbinden und Zugangsdaten zum Cluster

```
const {
     MongoClient
} = require('mongodb');

 const uri = "mongodb+srv://
              username:A2fPQYctCXDX7PQd
              @mongocluster.123.mongodb.net/
              test?retryWrites=true&w=majority";
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Liste der Datenbanken abfragen

```
async function main() {

    const client = new MongoClient(uri);

    async function listDatabases(client) {
        listDatabases = await client.db().admin().listDatabases();

        console.log("Databases:");
        listDatabases.databases.forEach(db => console.log(` - ${db.name}`));
    };

    try {
       await client.connect();      // Connect to the MongoDB cluster

       await listDatabases(client); // Make the appropriate DB calls

    } catch (e) {
        console.error(e);
    } finally {
        await client.close();
    }
}

main().catch(console.err);
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Eine Collection erstellen

```
const client = new MongoClient(uri);
try {
  await client.connect();

  const database   = client.db("sample_mflix");
  const collection = database.collection("movies");

  // Query for a movie that has the title 'The Room'
  const query = {
    title: "The Room"
  };

  const options = {
    // sort matched documents in descending order by rating
    sort: {
            "imdb.rating": -1
          },

    // Include only the `title` and `imdb` fields in the returned document
    projection: {
                  _id: 0,
                  title: 1,
                  imdb: 1
                },
    }
```

2026 Michael Reichart, copyleft CC BY-NC-SA

In der Collection nach Eigenschaft finden

```
// Find a Document
coll.findOne({ title: 'Hamlet' });

// Find Multiple Documents
coll.find({ year: 2005 });
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Ein Dokument in die Collection einfügen

```
// Insert a Document
coll.insert({ title: 'Jackie Robinson' });
```

2026 Michael Reichart, copyleft CC BY-NC-SA

ECMA Script

Some ECMA Script 6+

2026 Michael Reichart, copyleft CC BY-NC-SA

ECMAScript 6 (2015)

1. Let and Const
2. Arrow functions
3. Default parameters
4. for of loop
5. Spread attributes
6. Maps
7. Sets
8. Static methods
9. Classes

2026 Michael Reichart, copyleft CC BY-NC-SA

Let

```
// let is similar to var but let has scope.
// let is only accessible in the block level it is
defined.

if (true) {
 let a = 40;
 console.log(a); //40
}

console.log(a); // undefined
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Const

```
// Const is used to assign a constant value to the
variable.
// And the value cannot be changed. Its fixed.

const a = 50;
a = 60;        // shows error.

You cannot change the value of const.
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Arrow Function

```
// Old Syntax
function fn () {
 console.log("Hello World..!");
}


function fn () { . . .}
fn = () => { . . . }

// New Syntax
Let fn = () => {
 console.log("Hello World..!");
}

() => { console.log("Hello World..!"); }
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Arrow Function mit Default

```
let Func = (a, b = 10) => {
 return a + b;
}

Func(20); // 20 + 10 = 30
```

2026 Michael Reichart, copyleft CC BY-NC-SA

For of loop

```
// for..of iterates through list of elements (i.e) like
Array
// and returns the elements (not their index) one by one.

let arr = [2,3,4,1];

for (let value of arr) {
 console.log(value);
}

Output:
2
3
4
1
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Maps

```
// Map holds key-value pairs, define an own index. Indexes are unique in maps.

let map = new Map(); //Empty Map
let map = new Map([[1,2],[2,3]]); // map = {1=>2, 2=>3}

map.get(1)// 2
map.has(1);//return boolean value:  true/false
map.set(4,5); //{1=>2, 2=>3, 4=>5}

var isDeleteSucceeded = map.delete(1); //{ 2=>3, 4=>5}
console.log(isDeleteSucceeded); //true

map.clear(); //{}

console.log(map.size);//0

//For map: { 2=>3, 4=>5}
for (const item of map){
    console.log(item);    //Array[2,3]; //Array[4,5]
}

map.forEach((value, key) => console.log(`key: ${key}, value: ${value}`));
//key: 2, value: 3; //key: 4, value: 5
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Sets

```
var emptySet = new Set();
var exampleSet = new Set([1,2,3]);

var arr = Array.from(set);//[1,2,3]

console.log(set.has(0)); // boolean - false
console.log(set.has(1)); //true

set.add(3); //{1,2,3}
set.add(4); //{1,2,3,4}

set.delete(4); //{1,2,3}
set.clear(); //{}
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Static methods

```
class Example {
  static Callme() {
    console.log("Static method");
  }
}

Example.Callme();


Output:
Static method
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Classes

2026 Michael Reichart, copyleft CC BY-NC-SA

Grundlagen

```
class Sheep {
  constructor(name) {
    this.name = name;
  }

  bleat() {
    return `${this.name} sagt: Mäh!`;
  }
}

const shaun = new Sheep("Shaun");
console.log(shaun.bleat()); // "Shaun sagt: Mäh!"
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Sichtbarkeit

- public: this.name
  von überall zugänglich

- #private: this.#weight
  nur innerhalb der Klasse
  (echter Datenschutz,
  ES2022)

- _protected: this._farm
  nur eine Konvention, nicht
  von außen verwenden

2026 Michael Reichart, copyleft CC BY-NC-SA

Getter und Setter

```
class Sheep {
  #name;

  constructor(name) {
    this.#name = name;
  }

  get name() {
    return this.#name.toUpperCase();
  }

  set name(newName) {
    if (!newName || newName.trim() === "") {
      console.error("Fehler: Name darf nicht leer sein!");
      return;
    }
    this.#name = newName;
  }
}

const shaun = new Sheep("Shaun");
console.log(shaun.name); // "SHAUN"

shaun.name = "Dolly";
console.log(shaun.name); // "DOLLY"

shaun.name = ""; // Fehler: Name darf nicht leer sein!
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Typischer Fehler bei Hettern und Settern

```
class Sheep {
  #woolKg = 0;

  get wool() {                    // Getter heißt "wool" ...
    return this.#woolKg;
  }

  set woolKg(value) {             // ... Setter heißt "woolKg" –
FALSCH
    this.#woolKg = value;
  }
}

const dolly = new Sheep();
dolly.woolKg = 4;
console.log(dolly.woolKg); // undefined – es gibt keinen Getter
"woolKg"
console.log(dolly.wool);   // 4 – nur über den anderen Namen
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Setter im Konstruktor

```
class Sheep {
  constructor() {
    this.breed = "Merino"; // ruft den Setter auf
  }

  get breed() {
    return this._breed;
  }

  set breed(value) {
    this._breed = value;
  }
}

const dolly = new Sheep();
console.log(dolly.breed); // "Merino"
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Vererbung

```
class Sheep {
  constructor(name) {
    this.name = name;
  }

  bleat() {
    return `${this.name} sagt: Mäh!`;
  }
}

class Merino extends Sheep {
  constructor(name, woolKg) {
    super(name);
    this.woolKg = woolKg;
  }
}

class Lamb extends Sheep {
  constructor(name, ageMonths) {
    super(name);
    this.ageMonths = ageMonths;
  }
}

const lamm = new Lamb("Timmy", 4);
console.log(lamm.bleat()); // "Timmy sagt: Mäh!" – geerbt von Sheep
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Getter und Setter

```
class SheepModel {
    #name; // Private Variable

    constructor(name) {
        this.#name = name;
    }

    // 1. Der GETTER (Daten sicher nach draußen geben)
    get name() {
        // Hier könnte man den Namen manipulieren, z.B. immer in Großbuchstaben
        return this.#name.toUpperCase();
    }

    // 2. Der SETTER (Daten vor dem Speichern überprüfen)
    set name(newName) {
        // Validierung: Ein leerer Name ist nicht erlaubt!
        if (!newName || newName.trim() === "") {
            console.error("Fehler: Name darf nicht leer sein!");
            return;
        }
        this.#name = newName;
    }
}
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Getter und Setter

```
// --- Nutzung von außen ---
const sheep = new SheepModel("Shaun");

// Zugriff über den Getter (Achtung: KEINE Klammern () wie bei
Funktionen!)

console.log(sheep.name);
// Ausgabe: "SHAUN" (wurde durch den Getter groß geschrieben)

// Zugriff über den Setter (Sieht aus wie eine normale
Zuweisung)
sheep.name = "Dolly";

console.log(sheep.name);
// Ausgabe: "DOLLY"
// Test der Validierung im Setter: sheep.name = "";
// Konsole meldet: "Fehler: Name darf nicht leer sein!"
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Classes, Klasse als ES Modul

```
class Control {
  constructor(value = 'default') {
    this._myProperty = value;
  }
  get property() {
   return this._myProperty;
  }
  set myProperty(value) {
    value = value.trim();

    if (typeof value === 'undefined') {
      throw new Error('value is undefined');
    }

    this._myProperty = value;
  }
  myMethod = function () {
    ...
  }
}

export default new Control();
```

2026 Michael Reichart, copyleft CC BY-NC-SA

```
public
#private
_protected
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Klassen, Konstruktor,

```
class Shape {

    constructor (id, x, y) {
        this._id = id;
    }
    move (x, y) {
        this.x = x
        this.y = y
    }
    get id () {}
    set id () { return this._id; }
}

let myShape = new Shape('rect', 10, 10);
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Vererbung, Elternkonstruktor

```
class Rectangle extends Shape {
    constructor (id, x, y, width, height) {
        super(id, x, y);
        this._width  = width;
        this._height = height;
    }
}

class Circle extends Shape {
    constructor (id, x, y, radius) {
        super(id, x, y);
        this._radius = radius;
    }
}
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Getter, Setter und underscore-Schreibweise

```
class Shape {
    constructor () {
        this.type = "circle"
    }
    set type (value) {
        this._type = value
    }
    get type () {
        return this._type
    }
}
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Datenaustausch und Kommunikation zwischen Server und Client



fetch()

2026 Michael Reichart, copyleft CC BY-NC-SA

AJAX - ASYNCRONOUS JAVASCRIPT AND XML

- AJAX ist eine standardisierte Methode, um per Javascript Daten vom Server anzufordern.

```
let request = new XMLHttpRequest();

request.open(filename, 'get');
request.send();
_ _ _ _ _ _ _ _
request.addEventListener('readystatechange',
function(){ ... });
```

- AJAX basiert auf dem `XMLHttpRequest` Object.

2026 Michael Reichart, copyleft CC BY-NC-SA

AJAX - ASYNCRONOUS JAVASCRIPT AND XML

```
xhr = new XMLHttpRequest();
xhr.addEventListener('readystatechange', onReadyStateChange);
xhr.open("GET", _file);
xhr.send();

function onReadyStateChange() {
	switch (xhr.readyState) {
		case 0:
			console.log('there is no request');
			break;
		case 1:
			console.log('request opened');
			break;
		case 2:
			console.log('request sent');
			break;
		case 3:
			console.log('response first part ...');
			break;
		case 4:
			console.log('response more parts and finished!');
			if (xhr.status === 200 || xhr.status === 304) {
				_data = JSON.parse(xhr.response);
			}
			break;
		default:
			console.log('something strange happened!');
			break;
	}
}
```

2026 Michael Reichart, copyleft CC BY-NC-SA

fetch()(ES 6+)

```
fetch('data.json')
  .then(function (response) {
    if (response.ok)
      return response.json();
    else
      throw new Error('Kurse konnten nicht geladen werden');
  })
  .then(function (json) {
    // Hier Code zum einarbeiten der Kurse in die Anzeige
  })
  .catch(function (err) {
    // Hier Fehlerbehandlung
  });

// fetch basiert auf Promises,
// die ebenfalls in ES 6+ eingeführt wurden
```

2026 Michael Reichart, copyleft CC BY-NC-SA

fetch() einstellen

```
fetch( url, {

  method: 'POST',                       // *GET, POST, PUT, DELETE, etc.

  mode: 'cors',                         // no-cors, *cors, same-origin

  cache: 'no-cache',                    // *default, no-cache, reload, force-cache,
                                        //  only-if-cached

  credentials: 'same-origin',           // include, *same-origin, omit

  headers: {
     'Content-Type': 'application/json' // 'application/x-www-form-urlencoded',
  },

  redirect: 'follow',                   // manual, *follow, error

  referrerPolicy: 'no-referrer',        // no-referrer, *no-referrer-when-downgrade,
                                        // origin, origin-when-cross-origin,
                                        // same-origin, strict-origin,
                                        // strict-origin-when-cross-origin,
                                        // unsafe-url

  body: JSON.stringify(data)            // body data type must match "Content-Type" header
})
```

2026 Michael Reichart, copyleft CC BY-NC-SA

https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

https://dmitripavlutin.com/javascript-fetch-async-await/

Mehr über fetch()

2026 Michael Reichart, copyleft CC BY-NC-SA

Axios

2026 Michael Reichart, copyleft CC BY-NC-SA

Architektur

SOLID

2026 Michael Reichart, copyleft CC BY-NC-SA

SOLID - die Wartbarkeit von Software verbessern

+ Single Responsiblity
+ Open-Closed
+ Liskovsche Substitution
+ Interface Segregation
+ Dependency Inversion

2026 Michael Reichart, copyleft CC BY-NC-SA

„S“ wie „Single-Responsibility-Prinzip“

- „Es sollte nie mehr als einen Grund dafür geben, eine Klasse zu ändern.“ Robert C. Martin
- Jede Klasse hat nur genau eine fest definierte Aufgabe zu erfüllen. Wenn eine Klasse mehrere Verantwortungen zu tragen hat, führt das zu Schwierigkeiten bei zukünftigen Änderungen und das Fehlerrisiko steigt.
- Eine hohe Kohäsion wird angestrebt. Alle Methoden innerhalb einer Klassen haben einen starken gemeinsamen Bezug

2026 Michael Reichart, copyleft CC BY-NC-SA

„O“ wie „Open-Closed-Prinzip“

- Module sollten offen für Erweiterungen sein, aber geschlossen für Modifikationen.
- Klassen, Methoden, Module werden so entwickelt, dass sie einfach zu erweitern sind – ohne ihr Verhalten zu ändern.
- Beispiel Vererbung: Das Verhalten einer Klasse wird nicht verändert, erhöht aber trotzdem die Funktionalität der Software.
- Auch das Überschreiben von Methoden verändert nicht das Verhalten der Basisklasse, sondern nur die Methoden der abgeleiteten Klasse.

2026 Michael Reichart, copyleft CC BY-NC-SA

Barbara Liskov

„L“ wie „Liskovsches Substitutionsprinzip“

- „Sei q(x) eine Eigenschaft des Objektes x vom Typ T, dann sollte q(y) für alle Objekte y des Typs S gelten, wobei S ein Subtyp von T ist.“ Barbara Liskov

2026 Michael Reichart, copyleft CC BY-NC-SA

„I“ wie „Interface-Segregation-Prinzip“

- „Clients sollten nicht dazu gezwungen werden, von Interfaces abzuhängen, die sie nicht verwenden.“ Robert C. Martin
- Wie der Name erahnen lässt, geht es hierbei darum, Interfaces aufzuspalten beziehungsweise sie nicht unnötig groß zu machen.
- Ein Interface soll also so gestaltetet sein, dass die Anforderungen des Clients erfüllt werden können. Damit soll vermieden werden, dass ein Client mehrere Interfaces nutzen muss, die er für seine eigentliche Anforderung gar nicht benötigt werden.

2026 Michael Reichart, copyleft CC BY-NC-SA

„D“ wie „Dependency-Inversion-Prinzip“

- A. Module hoher Ebenen sollten nicht von Modulen niedriger Ebenen abhängen. Beide sollten von Abstraktionen abhängen.

- B. Abstraktionen sollten nicht von Details abhängen. Details sollten von Abstraktionen abhängen.“ Robert C. Martin

- Das Prinzip beschäftigt sich mit der Abhängigkeit von abgeschlossenen funktionalen Einheiten einer Software – also Modulen. In Modulen, die eine höhere Hierarchie innerhalb der Software aufweisen, werden generelle Abläufe beschrieben, die von spezielleren Modulen verwendet werden.

- Je niedriger das Modul innerhalb der Hierachie, desto spezifischere Probleme löst es.

- Wenn ein hierarchisch niedriges Modul von einem höherliegendem Modul abhängig ist, entsteht ein Problem. Änderungen in spezifischen Modulen ändern somit das Verhalten von höher liegenden und generellen Modulen, eine zyklische Abhängigkeit entsteht und die Architektur und das Design der Software werden unnötig komplex. Das Prinzip soll also eine Invertierung der Abhängigkeiten sicherstellen.

2026 Michael Reichart, copyleft CC BY-NC-SA

Webpack

2026 Michael Reichart, copyleft CC BY-NC-SA

```
npm install webpack webpack-cli --save-dev
npm install clean-webpack-plugin --save-dev
npm install html-webpack-plugin --save-dev
npm install css-loader --save-dev
npm install style-loader --save-dev
npm install  babel-loader @babel/core @babel/preset-
env webpack

npm install webpack webpack-cli clean-webpack-plugin
css-loader style-loader --save-dev
```

2026 Michael Reichart, copyleft CC BY-NC-SA

webpack.conf.js

```
/** Webpack Config */

// Import
const path = require('path');
const HtmlWebpackPlugin = require('html-webpack-plugin');
const { CleanWebpackPlugin } = require('clean-webpack-
plugin');

module.exports = {
    mode: 'development', // 'development', 'production'

    // To mix frontend and backend code:
    // instructs webpack to target a specific environment,
    // here 'node'. Defaults to "web", others would be
'electron', ....
    target: 'node',

    // Instructions, how to build the application
    entry: {
        index: './src/index.js',
        // other: './src/other.js'
    },

    // Instructions, how to export the application
    output: {
        path: path.resolve(__dirname, 'dist'),
        filename: '[name].js' // index.js, other.js
    },

    module: {
        rules: [{
                test: /\.css$/,
                use: [{
                        loader: 'style-loader'
                    },
                    {
                        loader: 'css-loader'
                    }
                ]
            },
            // {
            //     test: /\.js$/,
            //     exclude: /node_modules/,
            //     use: [{
            //         loader: 'babel-loader',
            //         options: {
            //             presets: ['@babel/preset-env']
            //         }
            //     }],
            // },
            // {}, for more loader, i.e. images, fonts,
sass ...
        ]
    },

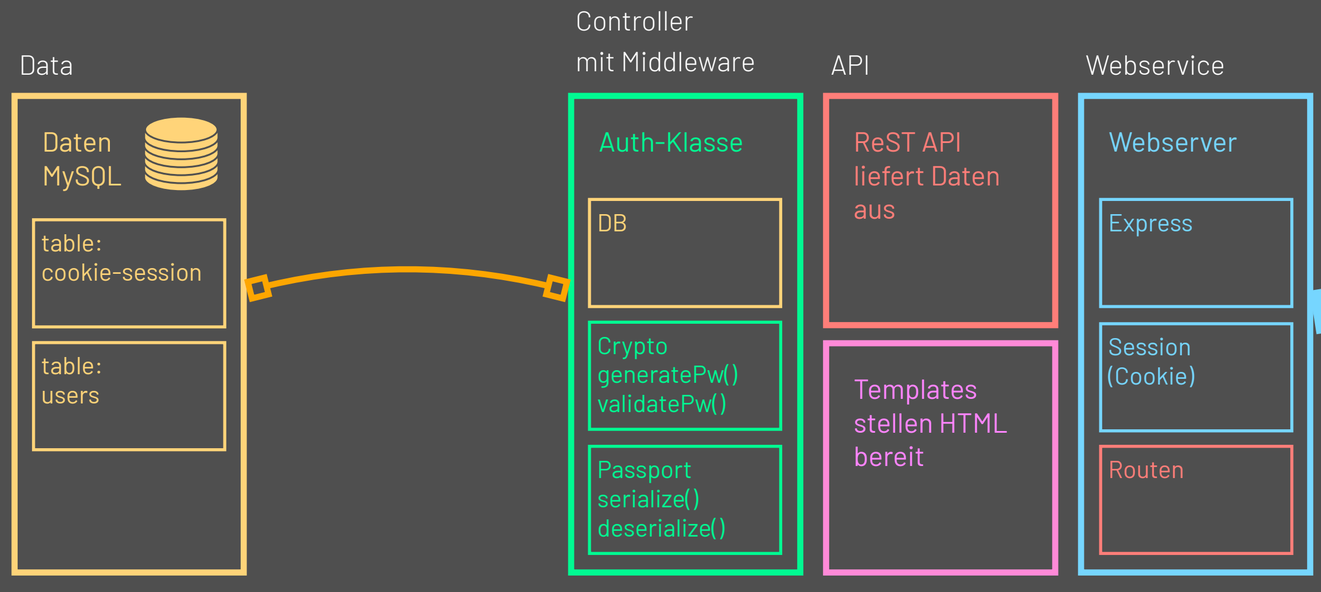
    plugins: [
        new CleanWebpackPlugin(),
        new HtmlWebpackPlugin({
            template: './src/index.html',
            filename: 'index.html'
        }),
    ],

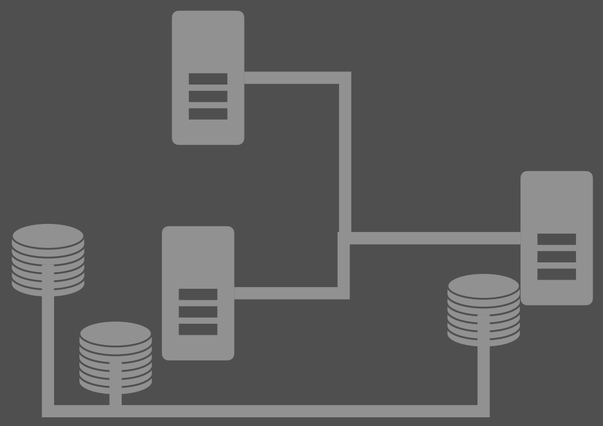
}
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Authentifizierung mit passport.js

2026 Michael Reichart, copyleft CC BY-NC-SA





Authentifizierung

2026 Michael Reichart, copyleft CC BY-NC-SA

Node-Applikationen skalieren

2026 Michael Reichart, copyleft CC BY-NC-SA

Was bedeutet Skalierung?

- Skalierung bezeichnet die Fähigkeit eines Systems, wachsende Anforderungen zu bewältigen - sei es durch mehr Benutzer, höheres Datenvolumen oder komplexere Anfragen.
- Das Ziel ist es, die Performance und Verfügbarkeit aufrechtzuerhalten, wenn die Last steigt.
- Es gibt zwei **Hauptaspekte**:
- **Performance**-Skalierung: Das System bleibt schnell und responsiv
- **Kapazitäts**-Skalierung: Das System kann mehr gleichzeitige Anfragen verarbeiten

2026 Michael Reichart, copyleft CC BY-NC-SA

Vertikale Skalierung (Scale Up)

- Bei der vertikalen Skalierung wird die **Hardware** des Servers aufgerüstet - mehr CPU, RAM oder bessere Festplatten.
- **Vorteile**:
  Einfach umzusetzen (Server upgraden)
  Keine Anpassungen am Code notwendig
  Keine Komplexität durch Verteilung
- **Nachteile**:
  Physische Grenzen der Hardware
  Teuer bei High-End-Hardware
  Single Point of Failure bleibt bestehen
  Downtime beim Upgrade
- Anwendungsfall: Geeignet als erste Maßnahme oder wenn die Anwendung nicht für Verteilung ausgelegt ist.

2026 Michael Reichart, copyleft CC BY-NC-SA

Horizontale Skalierung (Scale Out)

- Hier werden **mehrere Server-Instanzen** parallel betrieben, die sich die Last teilen.
- **Vorteile**:
  - Nahezu unbegrenzt skalierbar
  - Bessere Ausfallsicherheit (Redundanz)
  - Kostengünstiger (Commodity-Hardware)
  - Kein Downtime beim Skalieren
- **Nachteile**:
  - Komplexere Architektur erforderlich
  - Load Balancing notwendig
  - Session-Management herausfordernd
  - Mehr Infrastruktur zu verwalten

2026 Michael Reichart, copyleft CC BY-NC-SA

Horizontale Skalierung mit Cluster-Modulen für Multi-Core-Nutzung

```
const cluster = require(,cluster');
const http = require('http');
const numCPUs = require('os').cpus().length;
if (cluster.isMaster) {
    console.log(`Master ${process.pid} is running`);
    // Worker für jeden CPU-Kern erstellen
    for (let i = 0; i < numCPUs; i++) {
        cluster.fork(); }
        cluster.on('exit', (worker, code, signal) => {
        console.log(`Worker ${worker.process.pid} died`);
        cluster.fork(); // Neuen Worker starten
    });
} else {
    // Worker erstellen HTTP-Server http.createServer((req, res)
=> {
        res.writeHead(200);
        res.end('Hello World\n'); }).listen(8000);
        console.log(`Worker ${process.pid} started`);
}
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Load Balancing

- Ein Load Balancer verteilt eingehende Anfragen auf mehrere Server-Instanzen.
- Strategien:
  - *Round Robin*: Anfragen werden reihum verteilt
  - *Least Connections*: Server mit wenigsten aktiven Verbindungen bekommt die Anfrage
  - *IP Hash*: Basierend auf Client-IP für Session-Persistenz
  - *Weighted*: Server mit höherer Kapazität bekommen mehr Last

2026 Michael Reichart, copyleft CC BY-NC-SA

Beispiel mit nginx als Load Balancer

```
upstream node_backend {
    least_conn;
    server localhost:3000;
    server localhost:3001;
    server localhost:3002;
    server localhost:3003;
}
server {
    listen 80;
    location / {
        proxy_pass http://node_backend;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header
        Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Container-Orchestrierung (z.B. Kubernetes, Docker, Docker-Swarm ...)

+ Container-Orchestrierung ist die **automatisierte Verwaltung, Skalierung und Vernetzung von Containern** über mehrere Server hinweg.

+ Statt Container manuell zu starten, zu überwachen und zu verwalten, übernimmt ein Orchestrierungs-System diese Aufgaben intelligent und automatisch.

+ Container packen deine Node.js-Anwendung zusammen mit allen Abhängigkeiten in eine isolierte, portable Einheit:

+ Vorteile von Containern:
  - Identisches Verhalten in Entwicklung, Test und Produktion
  - Schneller Start (Sekunden statt Minuten)
  - Weniger Ressourcenverbrauch als VMs
  - Einfaches Deployment und Rollback
  - Isolation zwischen Anwendungen

2026 Michael Reichart, copyleft CC BY-NC-SA

Datenbank-Skalierung: Read Replicas

+ **Lesezugriffe** werden auf mehrere Datenbank-Kopien **verteilt**
+ **Schreibzugriffe** gehen auf den **Master**.

2026 Michael Reichart, copyleft CC BY-NC-SA

4. Datenbank-Skalierung

```
//const { Pool } = require(,pg');
// Master für Schreibzugriffe
const masterPool = new Pool({
    host: 'master-db.example.com', database: 'myapp', max: 20
});
// Replicas für Lesezugriffe
const replicaPools =
    [
        new Pool({
            host: 'replica1.example.com', database: ,myapp'
        }),
        new Pool({
            host: 'replica2.example.com', database: ,myapp'
        })
    ];
// Funktion zur Auswahl eines Replica
function getReplicaPool() {
    return replicaPools[Math.floor(Math.random() * replicaPools.length)];
}
// Verwendung async function getUser(id) {
    const pool = getReplicaPool();
    return await pool.query('SELECT * FROM users WHERE id = $1', [id]);
}
async function createUser(userData) {
    return await masterPool.query( 'INSERT INTO users (name, email) VALUES ($1, $2)',
                                   [userData.name, userData.email] );
}
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Sharding: Daten werden auf mehrere Datenbanken aufgeteilt (z.B. nach User-ID-Bereichen).

```
// Einfaches Sharding-Beispiel
class DatabaseSharding {
    constructor(shards) {
        this.shards = shards; // Array von DB-Pools
}
getShardForUser(userId) {
    const shardIndex = userId % this.shards.length;
    return this.shards[shardIndex];
}
async getUserData(userId) {
    const shard = this.getShardForUser(userId);
    return await shard.query('SELECT * FROM users
                              WHERE id = $1', [userId]);
    }
}
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Caching-Strategien

+ Caching speichert häufig abgerufene Daten temporär im schnellen Speicher (z.B. RAM via Redis)

+ Sie werden nicht mehr jedes Mal neu aus der langsamen Datenbank geladen.

+ Die Datenbankabfragen werden reduziert, die Response-Zeit verkürzt und die Gesamtlast des Systems (deutlich) gesenkt.

2026 Michael Reichart, copyleft CC BY-NC-SA

Caching Beispiel mit Redis

```
const redis = require(,redis');
const client = redis.createClient();
async function getCachedData(key, fetchFunction, ttl = 3600) {
    // Erst im Cache suchen
    const cached = await client.get(key);
    if (cached) {
        return JSON.parse(cached);
    }
    // Bei Cache-Miss: Daten holen und cachen
        const data = await fetchFunction();
        await client.setEx(key, ttl, JSON.stringify(data));
        return data;
    }
    // Verwendung
    app.get('/api/user/:id', async (req, res) => {
        const userId = req.params.id;
        const userData = await getCachedData(
            `user:${userId}`,
            () => database.getUserById(userId),
            1800 // 30 Minuten TTL );
        res.json(userData);
});
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Microservices-Architektur

+ Anstatt einer monolithischen Anwendung werden Funktionen in eigenständige Services aufgeteilt.
+ Services können unabhängig skaliert werden
+ Technologie-Stack pro Service wählbar
+ Einfacheres Deployment einzelner Komponenten
+ Bessere Fehler-Isolation

2026 Michael Reichart, copyleft CC BY-NC-SA

Web-Service für eine Bestell-Komponente

```
// User Service (Port 3001)
const express = require('express');
const app = express();

app.get('/users/:id', async (req, res) => {
  const user = await userDatabase.findById(req.params.id);
  res.json(user);
});

app.listen(3001);

// Order Service (Port 3002)
const orderApp = express();

orderApp.post('/orders', async (req, res) => {
  // Kommunikation mit User Service
  const userResponse = await fetch(`http://user-service:3001/users/${req.body.userId}`);
  const user = await userResponse.json();

  // Order erstellen
  const order = await createOrder(user, req.body);
  res.json(order);
});

orderApp.listen(3002);
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Message Queues & Asynchrone Verarbeitung

+ Message Queues ermöglichen es, zeitaufwändige Aufgaben (wie E-Mail-Versand oder Bildverarbeitung) asynchron im Hintergrund zu verarbeiten.

+ Die Aufgabe wird in eine Warteschlange gelegt und später von separaten Worker-Prozessen abgearbeitet.

+ Der User bekommt eine Response, **während die eigentliche Arbeit parallel erledigt** wird.

2026 Michael Reichart, copyleft CC BY-NC-SA

Beispiel mit RabbitMQ

```
const amqp = require('amqplib');

// Producer (API Server)
async function queueTask(taskData) {
  const connection = await amqp.connect('amqp://localhost');
  const channel = await connection.createChannel();
  const queue = 'tasks';

  await channel.assertQueue(queue, { durable: true });
  channel.sendToQueue(queue, Buffer.from(JSON.stringify(taskData)), {
    persistent: true
  });

  console.log('Task queued:', taskData);
  await channel.close();
  await connection.close();
}

// Consumer (Worker)
async function processQueue() {
  const connection = await amqp.connect('amqp://localhost');
  const channel = await connection.createChannel();
  const queue = 'tasks';

  await channel.assertQueue(queue, { durable: true });
  channel.prefetch(1);

  channel.consume(queue, async (msg) => {
    const task = JSON.parse(msg.content.toString());
    console.log('Processing task:', task);

    // Task verarbeiten
    await performHeavyTask(task);

    channel.ack(msg);
  });
}
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Content Delivery Networks (CDN)

- Statische Assets werden über geografisch verteilte Server ausgeliefert:
- Reduzierte Latenz für Endnutzer
- Entlastung der Origin-Server
- DDoS-Schutz

2026 Michael Reichart, copyleft CC BY-NC-SA

Konfiguration für CDN-Integration

```
app.use(express.static('public', {
    maxAge: '1d',
    setHeaders: (res, path) => {
        if (
            path.endsWith('.js')
            || path.endsWith('.css')) {
                res.setHeader(
                    Cache-Control',
                    public,
                    max-age=31536000');
            }
    }
}));

// In HTML-Templates
// <script src="https://cdn.example.com/app.js"></script>
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Best Practises für Skalierungen

- **Ein guter Anfang:**
- Cluster-Modul für Multi-Core-Systeme
- Caching für häufig abgerufene Daten
- Datenbankabfragen mit Indizes optimieren
- **Bei wachsender Last:**
- Load Balancer
- horizontale Skalierung mit Container-Orchestrierung (Kubernetes, Docker Swarm)
- Database Read Replicas
- **Für große Systeme:**
- Microservices-Architektur
- Message Queues für asynchrone Verarbeitung
- Auto-Scaling basierend auf Metriken
- Die richtige Skalierungsstrategie hängt von deinen spezifischen Anforderungen, Budget und technischen Möglichkeiten ab. Oft ist eine Kombination verschiedener Ansätze am effektivsten.
-

2026 Michael Reichart, copyleft CC BY-NC-SA

Funktionale Tests mit Jest

2026 Michael Reichart, copyleft CC BY-NC-SA

Jest-Vokabular: Mock

- Mock-Funktionen machen es einfach, die Verbindungen zwischen Code zu testen, indem sie die tatsächliche Implementierung einer Funktion auslöschen und die Aufrufe der Funktion (und die in diesen Aufrufen übergebenen Parameter) erfassen."
- Darüber hinaus können wir ein Mock verwenden, um zurückzugeben, was immer wir wollen, dass es zurückgibt. Dies ist sehr nützlich, um alle Pfade in unserer Logik zu testen, da wir kontrollieren können, ob eine Funktion einen richtigen oder falschen Wert zurückgibt oder sogar einen Fehler auslöst.

2026 Michael Reichart, copyleft CC BY-NC-SA

Jest-Vokabeln: Spy

- Erzeugt eine Mock-Funktion ähnlich jest.fn(), verfolgt aber auch Aufrufe von object[methodName]. Gibt eine Jest-Mock-Funktion zurück.
- Das bedeutet, dass sich die Funktion so verhält, wie sie es normalerweise tun würde - allerdings werden alle Aufrufe aufgezeichnet. Auf diese Weise können Sie überprüfen, ob eine Funktion so oft wie möglich aufgerufen wurde und die richtigen Eingabeparameter hatte.
- Im Folgenden finden Sie ein Beispiel, in dem wir überprüfen möchten, ob die Abspielmethode eines Videos das richtige Ergebnis liefert und mit den richtigen Parametern aufgerufen wird. Wir spionieren die Abspielmethode des Videoobjekts aus.
- Dann rufen wir die Wiedergabemethode auf und prüfen, ob der Spion aufgerufen wurde und ob das zurückgegebene Ergebnis korrekt ist. Ziemlich einfach! Zum Schluss müssen wir die Methode mockRestore aufrufen, um eine Attrappe auf ihre ursprüngliche Implementierung zurückzusetzen.

2026 Michael Reichart, copyleft CC BY-NC-SA

Jest-Grundlagen: `describe`-Blöcke und `it`- oder 'test'-Tests

- Ein **Beschreibungsblock** `describe`wird verwendet, um Testfälle in logischen Gruppen von Tests zu organisieren.
- Das Schlüsselwort `test` wird verwendet, um eine neue Testfalldefinition zu beginnen.
- Das Schlüsselwort ´it` ist ein Alias für das Schlüsselwort `test`.

2026 Michael Reichart, copyleft CC BY-NC-SA

`describe`, `it`

```
describe('Beverage()', () => {
   it('should be delicious', () => {
      expect(Beverage.delicious).toBeTruthy();
   });
});
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Jest-Grundlagen: Matcher

- Ein `matcher` wird zur Erstellung von Assertions in Kombination mit dem Schlüsselwort `expect` verwendet.

2026 Michael Reichart, copyleft CC BY-NC-SA

toBe, equal

```
test('two plus two is four', () => {
  expect(2 + 2).toBe(4);
});

test('adding positive numbers is not zero', () => {
  for (let a = 1; a < 10; a++) {
    for (let b = 1; b < 10; b++) {
      expect(a + b).not.toBe(0);
    }
  }
});

test('object assignment', () => {
  const data = {one: 1};
  data['two'] = 2;
  expect(data).toEqual({one: 1, two: 2});
});
```

2026 Michael Reichart, copyleft CC BY-NC-SA

instanceOf

```
it('should be instance of Car', () => {
   expect(newTruck()).toBeInstanceOf(Car);
});
```

2026 Michael Reichart, copyleft CC BY-NC-SA

null, undefined, true, false

```
test('null', () => {
  const n = null;
  expect(n).toBeNull();
  expect(n).toBeDefined();
  expect(n).not.toBeUndefined();
  expect(n).not.toBeTruthy();
  expect(n).toBeFalsy();
});

test('zero', () => {
  const z = 0;
  expect(z).not.toBeNull();
  expect(z).toBeDefined();
  expect(z).not.toBeUndefined();
  expect(z).not.toBeTruthy();
  expect(z).toBeFalsy();
});
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Zahlenwerte

```
test('two plus two', () => {
  const value = 2 + 2;
  expect(value).toBeGreaterThan(3);
  expect(value).toBeGreaterThanOrEqual(3.5);
  expect(value).toBeLessThan(5);
  expect(value).toBeLessThanOrEqual(4.5);

  // toBe and toEqual are equivalent for numbers
  expect(value).toBe(4);
  expect(value).toEqual(4);
});
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Fließkomma-Oprationen

```
test('adding floating point numbers', () => {
  const value = 0.1 + 0.2;
  // expect(value).toBe(0.3);     // geht nicht
  expect(value).toBeCloseTo(0.3); // geht
});
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Zeichenketten

```
test('there is no I in team', () => {
  expect('team').not.toMatch(/I/);
});

test('but there is a "stop" in Christoph', () => {
  expect('Christoph').toMatch(/stop/);
});
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Arrays

```
const shoppingList = [
  'diapers',
  'kleenex',
  'trash bags',
  'paper towels',
  'milk',
];

test('the shopping list has milk on it', () => {
  expect(shoppingList).toContain('milk');
  expect(new Set(shoppingList)).toContain('milk');
});
```

2026 Michael Reichart, copyleft CC BY-NC-SA

exeptions (Fehlerbehandlung)

```
function compileAndroidCode() {
  throw new Error('you use the wrong JDK!');
}

test('compiling android goes as expected', () => {
  expect(() => compileAndroidCode()).toThrow();
  expect(() => compileAndroidCode()).toThrow(Error);

  // You can use a string that must be contained
  // in the error message or a regexp
  expect(() => compileAndroidCode()).toThrow('you use the wrong JDK');
  expect(() => compileAndroidCode()).toThrow(/JDK/);

  // Or you can match an exact error message with regexp
  expect(() => compileAndroidCode())
              .toThrow(/^you use the wrong JDK$/); // Test fails

  expect(() => compileAndroidCode())
              .toThrow(/^you use the wrong JDK!$/); // Test pass
});
```

2026 Michael Reichart, copyleft CC BY-NC-SA

https://jestjs.io/docs/expect

Hier gibt es mehr dazu.

2026 Michael Reichart, copyleft CC BY-NC-SA

Tests einrichten und abbauen

2026 Michael Reichart, copyleft CC BY-NC-SA

Setup und Teardown (Generell)

```
describe('tests with database', () => {

  beforeEach(() => {
    initDB()
  })

  afterEach(() => {
    removeDB()
  })

  test('if country exists in database', () => {
    expect(isValidCountry('Belgium')).toBe(true)
  })

})
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Setup und Teardown (pro Test)

```
beforeAll(() => {
  return createDBConnection()
})

afterAll(() => {
  return destroyDBConnection()
})
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Typescript in nodejs

2026 Michael Reichart, copyleft CC BY-NC-SA

Typescript

- ein typisiertes "Superset" für Javascript, das nach Javascript transpiliert wird.
- **optionale** statische Typisierung. JavaScript Apps lassen sich daher einfach portieren.
- Unterstützt ECMAScript Features, die von aktuellen V8 Versionen noch nicht unterstützt werden. (siehe auch https://node.green/)
- Bessere OO: Klassen, aber auch Interfaces
- Bessere Zusammenarbeit mit IntelliSense oder Visual Studio Code.

2026 Michael Reichart, copyleft CC BY-NC-SA

Typisierung

- Strings haben eine `toLowerCase()` method, aber keine `parseInt()` Methode.
- Erweiterbare Typen: Eigene Typen können definiert werden.
- Aus `.js` wird `.ts`. Dann ist eine schrittweise Einführung in die Arbeit möglich.

2026 Michael Reichart, copyleft CC BY-NC-SA

```
$ npm install -g typescript

$ tsc --init
```

2026 Michael Reichart, copyleft CC BY-NC-SA

```
// helloPerson.ts

function sayHelloTo(person: string) {
    return `Hello ${person}!`
}

const myName = 'Michael'

console.log(sayHelloTo(myName))
```

2026 Michael Reichart, copyleft CC BY-NC-SA

```
project
|- dist
|- src
|   |- App.spec.ts
|   |- App.ts
|   |- index.ts
|
|- node_modules
|- .eslintrc.yaml
|- .gitignore
|- Dockerfile
|- package.lock.json
|- package.json
|- README.md
|- tsconfig.json
```

2026 Michael Reichart, copyleft CC BY-NC-SA

```
import * as express from 'express' // ES6!

class App {
  public express

  constructor () {
    this.express = express()
    this.mountRoutes()
  }

  private mountRoutes (): void {
    const router = express.Router()
    router.get('/', (request, response) => {
      response.json({
        message: 'Hello World!'
      })
    })
    this.express.use('/', router)
  }
}

export default new App().express
```

2026 Michael Reichart, copyleft CC BY-NC-SA

```
import app from './App'

const port = process.env.PORT || 3000

app.listen(port, (err) => { // arrow function
syntax!
  if (err) {
    return console.log(err)
  }

  return console.log(`server is now listening on $
{port}`)
})
```

2026 Michael Reichart, copyleft CC BY-NC-SA

```
{
  "compilerOptions": {
    "target": "es6",
    "module": "commonjs",
    "outDir": "dist",
    "sourceMap": true
  },
  "files": [
    "./node_modules/@types/mocha/index.d.ts",
    "./node_modules/@types/node/index.d.ts"
  ],
  "include": [
    "src/**/*.ts"
  ],
  "exclude": [
    "node_modules"
  ]
}
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Typisierte Variablen

```
let isDone: boolean = false;

let decimal: number = 6;
let hex: number = 0xf00d;
let binary: number = 0b1010;
let octal: number = 0o744;

let color: string = "blue";
let fullName: string = `Bob Bobbington`;
let sentence: string = `Hello, my name is ${ fullName
}.

let list: number[] = [1, 2, 3];
let list: Array<number> = [1, 2, 3];
```

2026 Michael Reichart, copyleft CC BY-NC-SA

React

2026 Michael Reichart, copyleft CC BY-NC-SA

„https://reactjs.de/artikel/react-tutorial-deutsch/“

–Paul Kögel, 09.11.2020

2026 Michael Reichart, copyleft CC BY-NC-SA

React

- React ist eine JavaScript-Bibliothek zum Erstellen von Benutzeroberflächen. (Facebook, BSD-Lizenz, 2013)
- Virtuelles DOM.
- Sehr gute Renderingperformanz.
- Modulare Komponentenarchitektur.
- React ist als Bibliothek konzipiert.

2026 Michael Reichart, copyleft CC BY-NC-SA

Reacts Bauteile

- Komponentenarchitektur.
- Virtueller DOM.
- Browserkompatibilität: synthetische Events, um jQuery-artig Browserunterschiede wegzuabstrahieren.

2026 Michael Reichart, copyleft CC BY-NC-SA

Komponenten Architektur

2021 Michael Reichart, copyleft CC BY-NC-SA

2026 Michael Reichart, copyleft CC BY-NC-SA

```
var ButtonCounter = React.createClass({
  getInitialState: function() { // (1)
    return({count: 0});
  },

  add: function() {
    this.setState(
      {count: this.state.count + 1}); // (2)
  },

  render: function() {
    return(
      <div>
        <button onClick={this.add}>
          +
        </button>
        <p>Count: {this.state.count}</p>
      </div>
    );
  }
});
```

2026 Michael Reichart, copyleft CC BY-NC-SA

JSx
Syntaxerweiterung für JavaScript

+ “If you’re going to hate on React for some reason, make it something other than JSX” — Alex Matchneer, Mitglied des Ember.js Core Teams.

© 2026 Michael Reichart, copyleft CC BY-NC-SA

```
<!DOCTYPE html>
<html>
<head>
<script src="//fb.me/react-with-addons-0.13.1.js"></
script>
  <meta charset="utf-8">
  <title>JS Bin</title>
</head>
<body>
  <div id='react-container'></div>
</body>
</html>
```

2026 Michael Reichart, copyleft CC BY-NC-SA

```
var ButtonCounter = React.createClass({
  getInitialState: function() { // (1)
    return({ count: 0 });
  },

  add: function() {
    this.setState(
      { count: this.state.count + 1 }); // (2)
  },

  style: function() { // (4)
    return { backgroundColor: this.props.colour };
  },

  render: function() {
    return(
      <div>
        <h1>{this.props.index + 1}. counter</h1> {/* (2) */}
        <button style={ this.style() }
                onClick={ this.add }>
          +
        </button>

        <p>
          Current count:
          <span>{this.state.count}</span>
        </p>
        <hr />
      </div>
    );
  }
});
```

2026 Michael Reichart, copyleft CC BY-NC-SA

```
var App = React.createClass({
  getInitialState: function() {
    return { colours: ['red', 'green', 'blue'] };
  },

  componentDidMount: function() {
    var that = this
    window.setInterval(function() {
      that.setState({ colours: [that.state.colours[1],
                                that.state.colours[2],
                                that.state.colours[0]] });
    }, 500);
  },

  buttons: function() {
    return this.state.colours.map(function(colour, index) { // (3)
      return(<ButtonCounter colour={ colour } index={ index } />);
    });
  },

  render: function() {
    return(
      <div>{ this.buttons() }</div>
    );
  }
});
```

2026 Michael Reichart, copyleft CC BY-NC-SA

```
React.render(<App />,
  document.querySelector('#react-container'));
```

2026 Michael Reichart, copyleft CC BY-NC-SA

Virtueller DOM

2021 Michael Reichart, copyleft CC BY-NC-SA

2026 Michael Reichart, copyleft CC BY-NC-SA

```
var Foo = React.createClass({
  render: function() {
    return(
      <div>
        <h1 className='mainHeadline'>Hallo
ReactJS.de</h1>
        <p>Lorem ipsum</p>
      </div>);
  }
});

var foo = new Foo();
console.log(JSON.stringify(foo.render(), null, 2));
```

2026 Michael Reichart, copyleft CC BY-NC-SA

```
"{
  "type": "div",
  "key": null,
  "ref": null,
  "_owner": null,
  "_context": {},
  "_store": {
    "props": {
      "children": [
        {
          "type": "h1",
          "key": null,
          "ref": null,
          "_owner": null,
          "_context": {},
          "_store": {
            "props": {
              "className": "mainHeadline",
              "children": "Hallo ReactJS.de"
            },
            "originalProps": {
              "className": "mainHeadline",
              "children": "Hallo ReactJS.de"
            }
          }
        },
        {
          "type": "p",
```


2026 Michael Reichart, copyleft CC BY-NC-SA